

Ingeniería de Servidores

Apuntes de Teoría

INGENIERÍA INFORMÁTICA

2025

ÍNDICE GENERAL

1	Capítulo 1 Introducción a la Ingeniería de Servidores	
1.1	¿Qué es un servidor?	5
1.2	Fundamentos de Ingeniería de Servidores	7
1.2.1	Prestaciones	7
1.2.2	Fiabilidad	7
1.2.3	Seguridad	8
1.2.4	Mantenimiento	8
1.2.5	Disponibilidad	8
1.2.6	Extensibilidad-expansibilidad	9
1.2.7	Escalabilidad	9
1.2.8	Coste	9
1.3	Introducción a la comparación de características entre servidores	9
1.3.1	Motivación: comparación de prestaciones	9
1.3.2	Tiempos de ejecución mayores/menores	10
1.3.3	Ganancia en velocidad (<i>Speedup</i> o aceleración)	10
1.3.4	Coste y relación prestaciones/coste	11
1.4	Ley de Amdahl	12
2	Capítulo 2 Componentes Hardware de un Servidor	
2.1	Placa Base	14
2.2	Fuente de alimentación, VMR y disipadores de calor	14
2.2.1	Fuente de alimentación	14
2.2.2	Módulo regulador de voltaje (VRM)	15
2.2.3	Disipadores de calor	15
2.3	CPU: zócalos, fabricantes y características	16
2.3.1	Evolución histórica de las CPUs	16
2.3.2	Procesadores AMD	16
2.3.3	IBM POWER	17
2.4	Memoria RAM dinámica	17
2.4.1	Ranuras para la memoria DRAM	17
2.4.2	Evolución histórica de las DRAM	18
2.4.3	Canales y bancos de memoria DRAM	19
2.4.4	Rangos de memoria DRAM (<i>memory ranks</i>)	19
2.5	Ranuras de Expansión	20
2.5.1	PCI y AGP	20
2.5.2	PCI-Express (PCIe)	21

2.5.3	Ventajas de usar una interfaz serie vs paralela	22
2.6	Almacenamiento y E/S	22
2.6.1	Almacenamiento permanente	22
2.6.2	Discos duros (HDD, <i>Hard Disk Drives</i>)	22
2.6.3	Unidades de estado sólido (SSD, <i>Solid State Drives</i>)	23
2.6.4	RAID (<i>Redundant Array of Independent Disks</i>)	23
2.6.5	Unidades ópticas	24
2.6.6	Unidades de cinta	24
2.6.7	SATA	24
2.6.8	SCSI y SAS	25
2.6.9	Conexión de múltiples unidades SAS	25
2.6.10	NVMe: Non-Volatile Memory Express	26
2.7	Conectores del panel trasero	26
2.7.1	Conectores internos	26
2.8	Chipset	27
2.9	La Carcasa (<i>Chassis</i>)	28

3

Capítulo 3

Monitorización de servicios y programas

3.1	Concepto de Monitor de Actividad	29
3.1.1	Clasificaciones de monitores de actividad	29
3.1.2	Atributos de un monitor	30
3.2	Monitorización a nivel de sistema	32
3.2.1	Directorio /proc (Linux)	32
3.2.2	Carga del sistema, según Linux	32
3.2.3	El monitor sar (<i>system activity reporter</i>)	34
3.2.4	Método USE (<i>Utilization, Saturation, Errors</i>)	37
3.2.5	Herramientas de monitorización de servidores distribuidos	37
3.3	Monitorización a nivel de aplicación	38
3.3.1	usr/bin/time	38
3.3.2	Profiler gprof	38
3.3.3	perf	40
3.3.4	valgrind	40
3.3.5	v-tune (Intel) y CodeXL (AMD)	40

4

Capítulo 4

Análisis comparativo del rendimiento

4.1	Referenciación (<i>benchmarking</i>)	41
4.1.1	Búsqueda de un buen índice de rendimiento para un componente concreto: CPU	41
4.1.2	Carga real	41
4.1.3	Clasificación benchmark	42
4.1.4	El paquete de microbenchmarks SPEC CPU	43
4.1.5	Benchmarks de sistema completo: TPC	44
4.2	Análisis de los resultados de un test de rendimiento	45
4.3	Comparación de prestaciones en presencia de aleatoriedad	46
4.3.1	Método 1	47
4.3.2	Método 2	47
4.3.3	Método 3	47
4.3.4	Ejercicio con tabla y cómo interpretarla	48
4.3.5	Estimar $\bar{d}_{real}$	48
4.4	Diseño de experimentos de comparación de rendimiento	48
4.4.1	Tipos de diseños experimentales	49

4.4.2	Diseños con un solo factor	49
-------	----------------------------------	----

5

Capítulo 5

Optimización del rendimiento de un servidor mediante análisis operacional

5.1	Introducción: Redes de Colas de Espera	50
5.1.1	El modelo de servidor central	51
5.1.2	Redes de colas cerradas	52
5.1.3	Redes de colas abiertas	52
5.1.4	Redes mixtas	52
5.2	Variables y leyes operacionales	53
5.2.1	Análisis operacional	53
5.2.2	Variables del servidor	53
5.2.3	Variables operacionales básicas	53
5.2.4	Variables operacionales deducidas	54
5.2.5	Variables operacionales de un servidor	55
5.2.6	Razón de visita y demanda de servicio	55
5.2.7	Leyes operacionales y equilibrio de flujo	56
5.3	Límites optimistas del rendimiento	58
5.3.1	Limitaciones en el rendimiento: cuello de botella	58
5.3.2	Saturación del servidor	59
5.3.3	Límites optimistas: redes abiertas	59
5.3.4	Límites optimistas: redes cerradas	60
5.3.5	Punto de saturación (<i>knee point</i>)	61
5.4	Técnicas de mejora	61
5.4.1	Sintonización o ajuste (<i>Tuning</i>)	61
5.4.2	Actualización y/o ampliación	61
5.5	Algoritmos de resolución de modelos de redes de colas	62
5.5.1	Resolución de Redes abiertas	62
5.5.2	Resolución de redes cerradas	63

6

Capítulo 6

Pliegos de prescripciones técnicas

6.1	Introducción	64
6.2	El pliego de condiciones	65
6.2.1	Pliego de Cláusulas Administrativas Particulares (PCAP)	65
6.2.2	Pliego de Prescripciones Técnicas (PPT)	66
6.3	Adjudicación de los contratos	67

INTRODUCCIÓN A LA INGENIERÍA DE SERVIDORES

1

1.1 ¿Qué es un servidor?

Definición 1.1.1 Sistema Informático (S.I)

Conjunto de elementos **hardware** (componentes físicos), **software** (componentes lógicos) y **peopleware** (recursos humanos) interrelacionados entre sí que permite obtener, procesar y almacenar información

Los Sistemas Informáticos pueden clasificarse según numerosos criterios, es común encontrar sistemas híbridos que no encajen en ninguna categoría. Por ejemplo, podríamos clasificarlos según su **paralelismo** de su arquitectura de procesamiento

S.I. según su uso

- **Uso general:** Son utilizados por un usuario para ejecutar muy diversas aplicaciones, como un
- **Uso específico:** Son utilizados para propósitos específicos, como los **sistemas empujados**, de una lavadora o los **servidores**, que sirven para proporcionar servicios.

Definición 1.1.2 Sistema empujado

Sistemas informáticos **acoplados** a otro dispositivo o aparato, diseñados para realizar una o algunas funciones dedicadas, frecuentemente con fuertes restricciones de tamaño, tiempo de respuesta (sistemas de tiempo real), consumo y coste. Suelen utilizar **microcontroladores**

Definición 1.1.3 Microcontrolador

Procesador, memoria e interfaces de comunicación en un ordenador muy pequeño diseñado para sistemas restrictivos

Definición 1.1.4 Servidores

Son sistemas informáticos que, formando parte de una red, **proporcionan servicios** a otros sistemas informáticos denominados **clientes**. Deben **pertenecer a una red** mediante la cual se conecta a sus clientes. Cualquier computador puede ser un servidor, incluso los **clusters de computadores** (muchos ordenadores juntos que se perciben como uno solo)

Nota ¿Puede un sistema empotrado ser un servidor?

La respuesta es ¡sí!. La diferencia entre un ordenador hecho para un propósito específico y uno que es de uso general, es que las especificaciones y la arquitectura están especialmente diseñadas para esa tarea.

Tipos de servidores

- **Servidor web:** procesa solicitudes realizadas a través de HTTP o HTTPS (Hypertext Transfer Protocol Secure).
- **Servidor de archivos:** permite el acceso remoto a archivos almacenados en él o directamente accesibles por este.
- **Servidor de base de datos:** provee servicios de base de datos a otros programas u otras computadoras.
- **Servidor de comercio-e:** facilita y procesa transacciones comerciales. P.ej. valida al cliente, procesa un pago, genera consultas al servidor de bases de datos...
- **Servidor DHCP** (*Dynamic Host Configuration Protocol*): almacena, envía, recibe, re-enruta y realiza otras operaciones relacionadas con e-mail para los clientes de la red.
- **Servidor DNS** (*Domain Name System*): asigna dinámica-mente una dirección IP y otros parámetros de configuración de red a cada dispositivo que entra en una red.
- **Servidor de impresión:** controla una o más impresoras y acepta trabajos de impresión de otros clientes de la red.

S.I. según la arquitectura de servicio

- **Aislado:** Cada componente del sistema **funciona independientemente** sin depender de otros componentes internos ni externos. Cada sistema aislado tiene su propia base de datos, servidor y lógica de negocio.
- **Cliente/servidor:** Los **clientes solicitan recursos** a través de una conexión a un **servidor** que maneja las peticiones, proporcionan respuestas y acceso a la información almacenada en su base de datos.
 - **Varios niveles:** El servidor se puede subdividir en varios niveles de servidores más sencillos con funcionalidades diferentes en cada nivel. Lo que permite la distribución de carga entre los diversos servidores.
- **Cliente-cola-cliente:** Varios clientes **comparten una misma infraestructura de servidor**, pero operan **independientemente** entre sí. Cada cliente tiene acceso a la base de datos y puede realizar solicitudes sin depender de los demás. Las solicitudes se encolan y distribuyen en los clientes por un servidor.

Un servidor, como todo sistema informático, tiene elementos físicos (hardware), lógicos (software) y humanos (peopleware). Pero los elementos más importantes para él son los siguientes:

1.2.1. Prestaciones

Definición 1.2.1 Prestaciones (*Performance*)

Es la medida de la **velocidad** con la que **se realiza** determinada **carga** (*load/workload*) de trabajo.

Hay dos medidas fundamentales en las prestaciones de un servidor:

- 1 **Tiempos de respuesta** (*response time*) o **latencia** (*latency*): Tiempo transcurrido desde que se solicita una tarea al servidor o a un componente y la finalización de dicha tarea.
- 2 **Productividad** (*throughput*) o **ancho de banda** (*bandwidth*): Cantidad de trabajo por unidad de tiempo realizado por el servidor o por un componente.

Nota Sobre la productividad y el tiempo de respuesta

La productividad se estanca cuando llegan más tareas de las que puedo gestionar por unidad de tiempo. Sin embargo, el tiempo de respuesta siempre es creciente, puesto que cuanta más gente haya pidiendo un servicio, más cola hay para solicitarlo y más tiempo hay que esperar para obtenerlo.

Para mejorar nuestras prestaciones, podemos cambiar o ajustar el **hardware**, el **Sistema operativo** y las **aplicaciones** según nuestras necesidades.

1.2.2. Fiabilidad

Definición 1.2.2 Fiabilidad (*Reliability*)

Probabilidad de que el servidor **funcione correctamente** durante un determinado intervalo de tiempo. Un servidor 100 % fiable debe ser tolerante tanto a **fallos físicos** (permanentes y temporales) como a **fallos software**. Se suele medir en fallos/segundos (FIT) o en tiempo medio hasta el fallo (MTTF/MTBF).

Para mejorarla, podemos utilizar **sumas de comparación** de bits (*checksums*) para **detectar errores** o comprobar paquetes, **sistemas de alimentación ininterrumpida** (SAI/*UPS*) para **evitar cortes de luz** que apaguen nuestro sistema o **sistemas redundantes de discos** (*RAID*) (o de alimentación, de red, etc.) que **detecten fallos de componentes y reaccionen** ante ellos.

1.2.3. Seguridad

Características

- ❶ **Confidencialidad:** Evitar la incursión de individuos no autorizados.
- ❷ **Integridad:** Evitar la corrupción o alteración no autorizada de datos.
- ❸ **Anti-ataques:** Evitar las interferencias que impidan el acceso a los recursos.

Para ello podemos incluir anti-virus, *firewalls*, encriptación de datos y otras medidas que veremos más adelante.

1.2.4. Mantenimiento

Definición 1.2.3 Mantenimiento (*Maintenance, support*)

Todas las acciones que tienen como objetivo prolongar el funcionamiento correcto del sistema.

Estas acciones incluyen, entre otros: Sistemas operativos actualizados, copias de seguridad y automatización de tareas.

1.2.5. Disponibilidad

Definición 1.2.4 Disponibilidad (*Availability*)

% de tiempo en el que el servidor está operativo (respondiendo a las peticiones), independientemente de que las respuestas a dichas peticiones sean correctas o no.

Definición 1.2.5 Tiempo de inactividad (*Downtime*)

Cantidad de tiempo en el que el servidor no está disponible

Hay dos tipos de inactividad:

- ❶ **Planificada:** Para labores de mantenimiento o actualización del servidor.
- ❷ **No planificada:** Surgen de algún evento no esperado tales como fallos del sistema o ataques.

Podemos mejorarla insertando componentes en caliente (*hot-pluggin*), reemplazando componentes en caliente (*hot-swapping*) o usando cualquier medida que aumente la fiabilidad.

Nota Fiabilidad vs Disponibilidad

Imaginemos que nuestro ordenador explota - NO es fiable NI disponible.

Imaginemos que nuestro ordenador está encendido pero devuelve $2+2=9$ - NO es fiable pero SI disponible.

Imaginemos que nuestro ordenador está encendido y además, devuelve $2+2=4$ - SI es fiable Y disponible.

1.2.6. Extensibilidad-expansibilidad

Definición 1.2.6 Extensible/Expansible

Hace referencia a la facilidad que ofrece el sistema para **aumentar** sus características o recursos

Esto se puede conseguir teniendo conectores libres, racks de servidores, SO modulares o interfaces de E/S estándar. Cualquier mejora que permita **escalabilidad**, **aumentaría la expansibilidad**

1.2.7. Escalabilidad

Definición 1.2.7 Escalabilidad

Facilidad que ofrece el sistema para poder **aumentar** de forma **significativa** sus características o recursos para enfrentarnos a un aumento significativo de la carga

Esto se logra con servidores modulares (*clusters*), arquitecturas por capas, programación paralela, etc. **Todos** los sistemas **escalables son extensibles** pero **no al revés**.

Nota Expansible vs Escalable

En resumen: Expansible es que "puedo ponerle cosas", escalable es que funciona igual independientemente de cuántos usuarios pidan su servicio.

Un ejemplo de expansible pero no escalable sería añadir dos fuentes de alimentación, pues eso no afecta a la escalabilidad, sino a la disponibilidad.

1.2.8. Coste

Es el presupuesto que tienes para tu servidor, que no es infinito. Puedes reducirlo con código abierto o reduciendo costes de la electricidad. Todos los elementos de un sistema informático cuestan dinero y hay que tenerlos en cuenta cuando lo creamos.

1.3

Introducción a la comparación de características entre servidores

1.3.1. Motivación: comparación de prestaciones

El computador de mejores prestaciones (el más rápido), para un determinado conjunto de programas, será aquel que ejecuta dicho conjunto de programas en el **tiempo más corto**. Cuando comparamos prestaciones, se suele indicar cuántas **veces es más rápido** y qué **tanto por ciento de mejora** aporta.

1.3.2. Tiempos de ejecución mayores/menores

Fórmula 1.3.1 A es X veces B

Sea t_A = tiempo de ejecución de un determinado programa en la máquina A (ídem para t_B).
En general, t_A es t_A/t_B veces t_B

- **Ejemplo:** $t_A = 10s, t_B = 5s \rightarrow t_A$ es $10/5 = 2$ veces t_B (el doble).
- Igualmente, en el ejemplo anterior: t_B es $t_B/t_A = 5/10 = 0,5$ veces t_A (la mitad)

Fórmula 1.3.2 Cambio relativo

El **cambio relativo de t_A con respecto a t_B** , $\Delta t_{A,B}(\%)$, viene dado por:

$$t_A = t_B + \frac{\Delta t_{A,B}(\%)}{100} \times t_B$$

De donde: $\Delta t_{A,B}(\%) = \frac{t_A - t_B}{t_B} \times 100 = \left(\frac{t_A}{t_B} - 1\right) \times 100$

- En el ejemplo anterior, el cambio relativo de t_A con respecto a t_B es 100 %
- Igualmente, el cambio relativo de t_B con respecto a t_A es -50 %

1.3.3. Ganancia en velocidad (*Speedup* o aceleración)

Fórmula 1.3.3 Ganancia (*Speedup*) de A respecto a B

Para un programa, se define la **ganancia en velocidad** de la máquina A con respecto a la máquina B como:

$$S_B(A) = \frac{v_A}{v_B} = \frac{\text{Trabajo}/t_A}{\text{Trabajo}/t_B} = \frac{1/t_A}{1/t_B} = \frac{t_B}{t_A}$$

El cambio relativo de v_A con respecto a v_B (% de cambio en velocidad al reemplazar la máquina A por la B) viene dado por:

$$\Delta v_{A,B}(\%) = \frac{v_A - v_B}{v_B} \times 100 = \left(\frac{v_A}{v_B} - 1\right) \times 100 = (S_B(A) - 1) \times 100$$

Nota ¿20 % más rápida significa que la otra es 20 % más lenta?

¡NO!, los % son relativos a cada máquina. Por ejemplo:

$$t_A = 36s, t_B = 45s$$

$$S_B(A) = \frac{45}{36} = 1,25 \text{ y } (1,25 - 1) \times 100 = 25 \%. \text{ A es un 25 \% más rápida que B.}$$

$$S_A(B) = \frac{36}{45} = 0,8 \text{ y } (0,8 - 1) \times 100 = -20 \%. \text{ B es un 20 \% más lenta que A.}$$

1.3.4. Coste y relación prestaciones/coste

Supongamos que, siguiendo el ejemplo anterior ($t_A = 36s$ y $t_B = 45s$):

- El computador **A** cuesta **625€**.
- El computador **B** cuesta **550€**.

El computador A es $625/550 = 1,14$ veces **más caro** que el B (un 14 %). ¿Cuál ofrece mejor relación prestaciones/coste para nuestro programa?

Fórmula 1.3.4 Relación prestaciones/coste

$$\frac{Prestaciones_A}{Costes_A} = \frac{v_A}{Coste_A} \propto \frac{1/t_A}{Coste_A}$$

$$\frac{Prestaciones_A}{Costes_A} = \frac{\frac{1}{36}}{625} = 4,4 \times 10^{-5} s^{-1}/euros$$

$$\frac{Prestaciones_B}{Costes_B} = \frac{\frac{1}{45}}{550} = 4,0 \times 10^{-5} s^{-1}/euros$$

Si usamos esta fórmula, tenemos que buscar el número más grande, puesto es el que mayor proporción coste/prestaciones tiene.

Fórmula 1.3.5 Relación prestaciones/costes de A es X veces B

El resultado de esta fracción es cuántas veces mayor es para nuestro programa A. Si queremos el %, necesitamos restarle 1 y multiplicarlo por 100.

$$\frac{Prestaciones_A}{Costes_A} \div \frac{Prestaciones_B}{Costes_B}$$

El computador A presenta una “mejor” relación prestaciones/costes que el B (1.1 veces mayor = 10 % mayor) para nuestro programa

$$\frac{Prestaciones_A}{Costes_A} \div \frac{Prestaciones_B}{Costes_B} = \frac{4,4 \times 10^{-5}}{4,0 \times 10^{-5}} = 1,1$$

1.4

Ley de Amdahl

Terminología

- T_o : **Tiempo original** en ejecutar cierto proceso.
- k : **Cuántas veces mejoramos** el componente que nos interesa. $(k - 1) \cdot 100\%$ más rápido.
- f : **Fracción del tiempo que se usa** el componente. Si es $f = 1$, se usa todo el tiempo.
- $1 - f$: **Fracción del tiempo que NO se usa** el componente.
- T_m : **Tiempo mejorado** después de mejorar el componente.

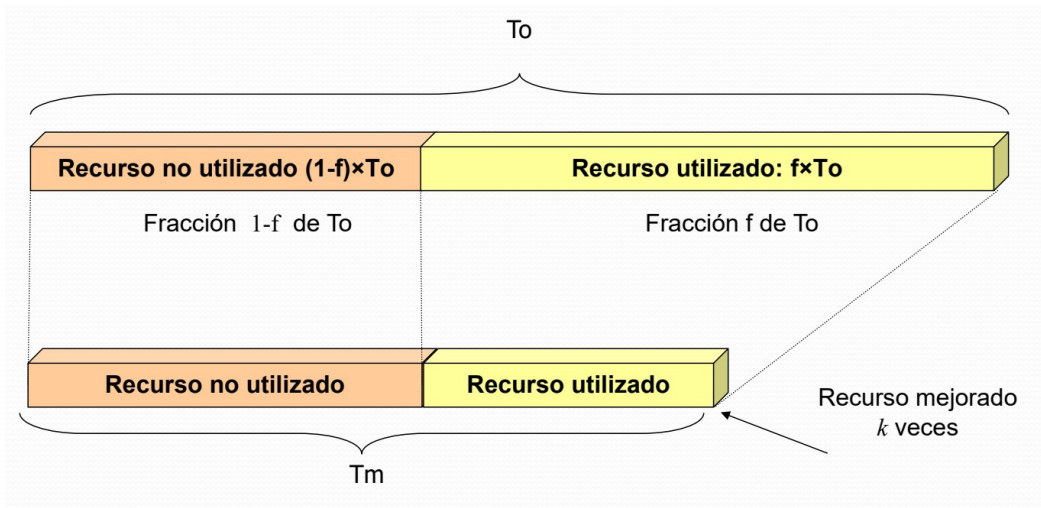


Figura 1.1: Tiempo original (T_o) vs tiempo mejorado (T_m)

Fórmula 1.4.1 Ley de Amdahl

$$T_m = (1 - f) \times T_o + \frac{f \times T_o}{k}$$

$$S \equiv S_{original}(mejorado) = \frac{v_m}{v_o} = \frac{T_o}{T_m} = \frac{T_o}{(1 - f) \times T_o + \frac{f \times T_o}{k}} = \frac{1}{1 - f + f/k}$$

Nota Casos particulares

- Si $f = 0 \Rightarrow S = 1$, significa que ninguna fracción del programa utiliza el componente que vamos a mejorar, así que no ganaremos nada
- Si $f = 1 \Rightarrow S = k$, significa que el sistema mejora tanto como el componente, puesto que lo usamos en la totalidad del tiempo
- Si $k \rightarrow \infty$, significa que estamos mejorando TODO LO POSIBLE el componente, que es $\frac{1}{1-f}$

Las veces que mejoramos el componente (k) **afectan más cuanto más tiempo utilizemos el componente** en nuestro programa. Evidentemente, si solo lo empleamos unos microsegundos, la ganancia será despreciable y deberíamos centrar nuestros esfuerzos en otros componentes más solicitados.

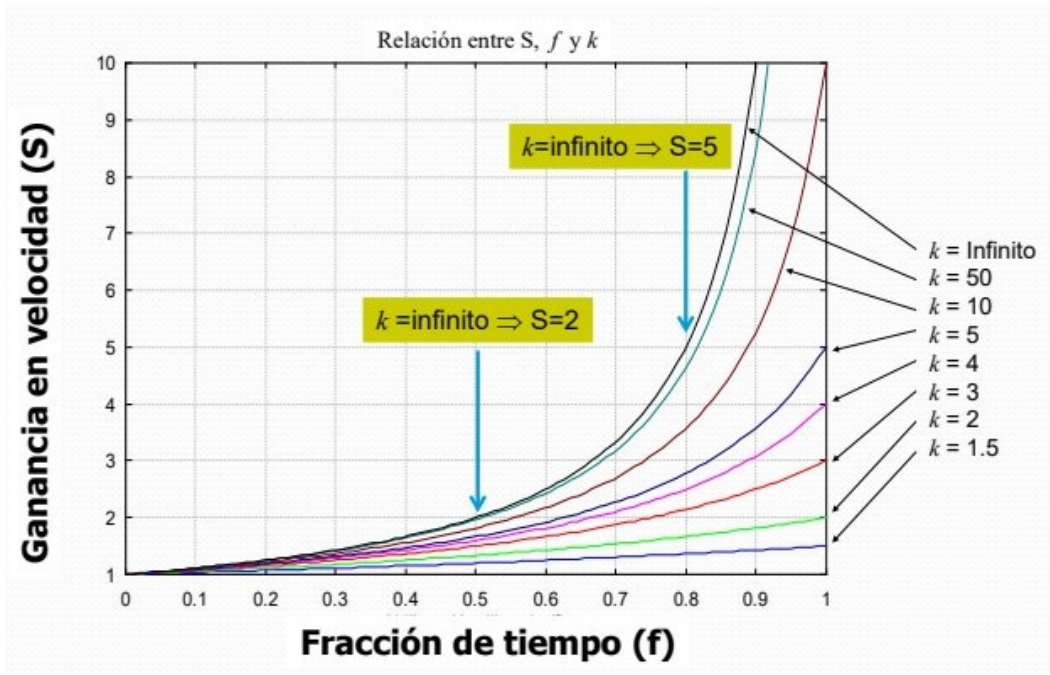


Figura 1.2: Relación entre ganancia y fracción del tiempo que usa el componente

Fórmula 1.4.2 Generalización de la Ley de Amdahl

Es la misma fórmula, pero haces un sumatorio de las f afectadas para calcular el tiempo que no se toca ($1 - f$) y realizas un sumatorio de los componentes mejorados con sus respectivas k :

$$\frac{1}{(1 - \sum_{i=1}^n f_i) + \sum_{i=1}^n \frac{f_i}{k_i}}$$

En otras palabras: "Uno menos todas las fracciones de tiempo que mejoro + todas las fracciones que mejoro partidas de su respectiva mejora"

COMPONENTES HARDWARE DE UN SERVIDOR

2

2.1 Placa Base

Definición 2.1.1 Placa base

Una **placa base** (o placa madre, motherboard, mainboard) es la tarjeta de circuito impreso (**PCB**, *Printed Circuit Board*) principal de un computador. En ella se conectan los **componentes hardware** del computador y contiene diversos **conectores** para añadir distintos periféricos adicionales.

- Está hecha de una lámina de un **substrato no conductor** sobre la que se extienden pistas de cobre.
- Son **multi-capa**, a través de agujeros conectamos las pistas (circuitos) de una capa a otra.
- Se fabrican con **distintos tamaños** y formas.

2.2 Fuente de alimentación, VMR y disipadores de calor

2.2.1. Fuente de alimentación

Definición 2.2.1 Fuente de alimentación

Convierte corriente alterna en continua y alimenta la placa base y los periféricos. Se mide en potencia (W)



Figura 2.1: Fuente de alimentación. De izquierda a derecha: fuente de PC, fuente de servidor (tiene un mango para sacarla), enchufes de alimentación

2.2.2. Módulo regulador de voltaje (VRM)

Definición 2.2.2 VRM

Adapta la tensión continua de la fuente de alimentación (5V, 12V) a las tensiones continuas **menores** que necesitan los diferentes elementos de un computador, dándoles **estabilidad**.

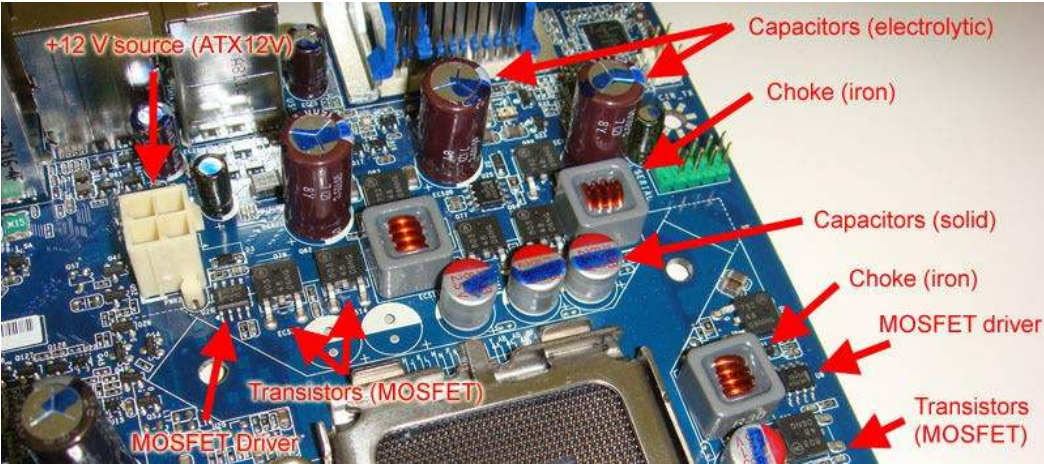


Figura 2.2: Placa base con cilindros que son los VRM

2.2.3. Disipadores de calor

- **Pasivos**, enfrían por el metal que está sobre la CPU.
- **Activos**, se mueven activamente para enfriar el CPU (ventiladores, refrigeración líquida)



Figura 2.3: A la izquierda disipador pasivo, debajo un pasivo + un activo, a la derecha activos (Ventilador de chasis) y conector del ventilador a la placa.

2.3

CPU: zócalos, fabricantes y características

Definición 2.3.1 Zócalos para el microprocesador (CPU Sockets)

Facilitan la conexión entre el microprocesador y la placa base de tal forma que el microprocesador pueda ser reemplazado sin necesidad de soldaduras.

Los zócalos para micros con un número grande de pines suelen ser del tipo PGA-ZIF (pin grid array - zero-insertion force) o LGA (land grid array), que hacen uso de una pequeña palanca (PGA-ZIF) o una pequeña placa de metal (LGA) para fijar el micro al zócalo. De esta forma, se minimiza el riesgo de que se doble alguna patilla durante el proceso de inserción.

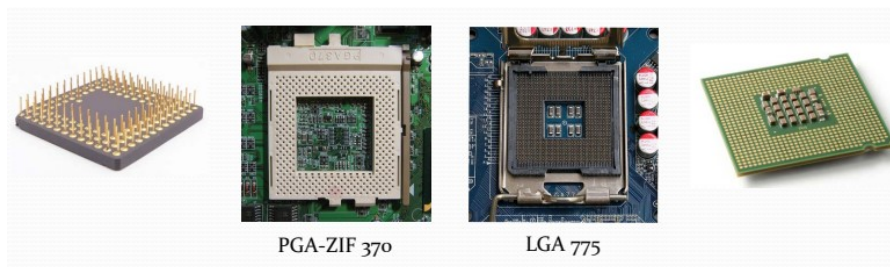


Figura 2.4: A la izquierda: zócalo PGA con una palanquita y pines que se insertan. A la derecha: zócalo LGA con una placa de metal que se baja sobre el procesador y los chips en el centro. Sin pines.

2.3.1. Evolución histórica de las CPUs

El objetivo, al principio, era aumentar la frecuencia de la CPU. Pero se alcanzó lo que se llama barrera térmica. Este término significa que gastas más energía en enfriar el microprocesador que lo que ganas aumentando los GHz. Esto ocurrió sobre 2003. Como no podían aumentar la frecuencia, empezaron a aumentar el número de cores (núcleos).

Diferencias entre microprocesador para servidores y para PC

- Más núcleos
- Suelen multiprocesador (2 o más CPU por placa)
- Más caché
- RAM con ECC, fiable.
- Más canales de RAM, más líneas de E/S.
- Más controles de calidad (funciona 24/7)
- Más tecnologías propias de servidores como encriptación, virtualización o multi-hebra.

2.3.2. Procesadores AMD

Originalmente denominada Opteron, el primer procesador de esta gama fue el primero con instrucciones x86-64. También fueron los primeros x86 con dos núcleos. Ahora los llama EPYC, tienen varios chips de procesamiento y otros más baratos para E/S. Cada chip tiene 8 cores con más caché que un PC de sobremesa.

2.3.3. IBM POWER

Servidores de gama alta de muy altas prestaciones por vatio, disponibilidad y fiabilidad. Se usan mainframes empresariales. Suelen desarrollar tecnologías antes de que estén disponibles para el consumo de PC de sobremesa.

2.4 Memoria RAM dinámica

2.4.1. Ranuras para la memoria DRAM

Existe una jerarquía de memoria, que explica que cuanto más veloz es, más cara y más pequeña también. Según esta jerarquía, tenemos dos tipos de RAM.

- **DRAM**, dinámica, la que solemos llamar RAM y es más barata
- **SRAM**, la del procesador (caché), es más cara y necesita más componentes para funcionar. Es mucho más rápida también.

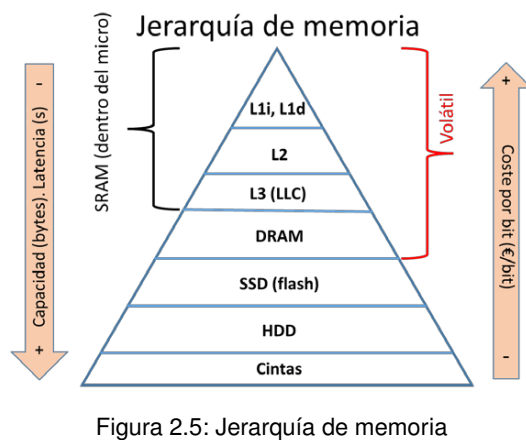


Figura 2.5: Jerarquía de memoria

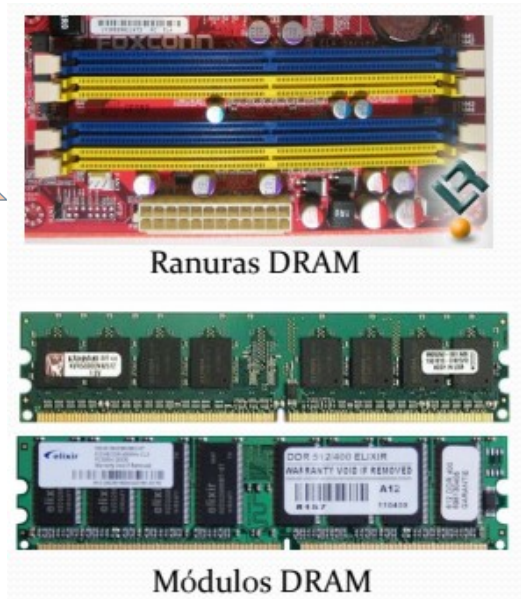


Figura 2.6: Ranuras y módulos DRAM

2.4.2. Evolución histórica de las DRAM

De aquí lo importante es saber que SDRAM $\neq$ SRAM, sino que es una RAM síncrona con la CPU y que los módulos que se utilizan ahora son DDR (Double Data Rate) y el más común ahora es el DDR4. Han aumentado de capacidad y siempre que sale un nuevo modelo reemplaza en el mercado a la mayoría del anterior.

La evolución de la ram ha cambiado en lo siguiente:

- Más contactos
 - Menos voltaje
 - Más bus de datos
 - Más ancho de banda
- PC y portátiles
 - **DIMM** o U-DIMM (la normal)
 - **SO-DIMM** (más chica, menos contactos, para portátiles)
 - Servidores
 - **EU-DIMM**: U-DIMM (normal) pero con código corrector de errores ECC.
 - **R-DIMM**: Guarda en un registro operaciones para módulos grandes
 - **LR-DIMM**: Tiene mayor latencia que R-DIMM y es aún más grande. Guarda tanto operaciones como datos en un buffer.

Tipos de DIMM (dual in line memory module)

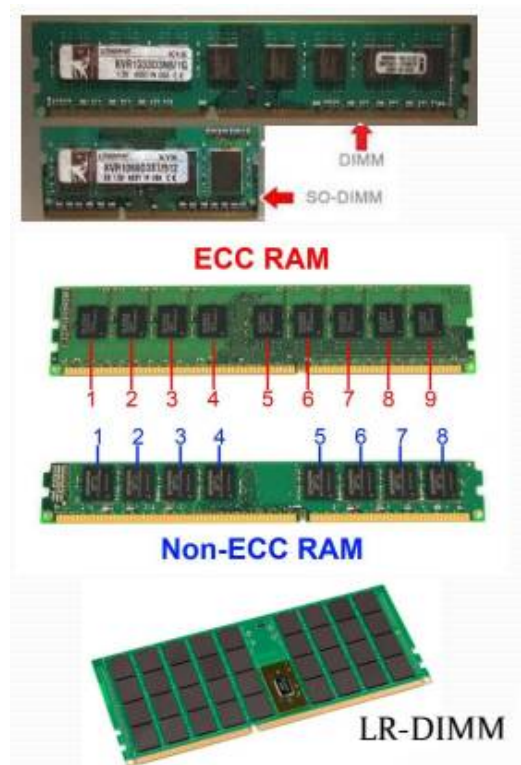


Figura 2.7: Tipos de RAM

2.4.3. Canales y bancos de memoria DRAM

Definición 2.4.1 Canal de memoria DRAM

Conexión directa del procesador y la RAM

Definición 2.4.2 Banco de memoria DRAM

Agrupación de ranuras que usan **el mismo canal**

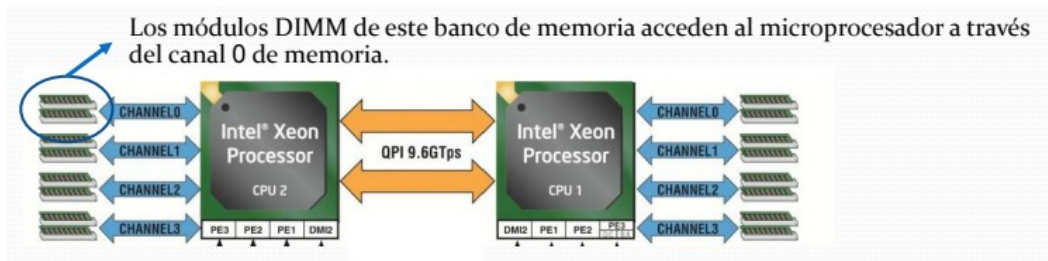


Figura 2.8: Canales y bancos de memoria. Las dos DRAM rodeadas por el circulito son un banco, que acceden a la CPU por el canal 0.

2.4.4. Rangos de memoria DRAM (*memory ranks*)

Definición 2.4.3 Rango de memoria

Se refiere al número de chips de memoria que lleva cada módulo de RAM y si están en una sola cara o en ambas. La notación es 1Rx8: Módulo de 1 rango con chips de 8 bits. En este caso sería 8 bits x 8 chips.

Tipos de rangos de memoria

- **Single Rank y Single Sided:** un solo rango, un solo lado del módulo de RAM con chips.
- **Single Rank y Dual Sided:** Un rango pero dos lados donde hay chips.
- **Dual Rank y Dual Sided:** Dos rangos diferentes y dos lados donde hay chips.

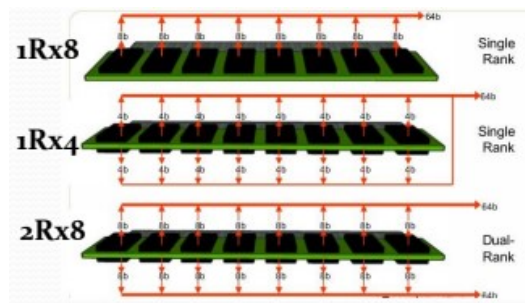


Figura 2.9: Ejemplos de rangos de memoria. Tenemos módulos que tienen chips solo por un lado (1Rx8) o por los dos (1Rx4 y 2Rx8)

Nota MB vs MiB

Cuando encontramos MB significa 1000 KB (Kilo Bytes) pero si encontramos MiB significa 1024 kB/KiB (Kili Bytes).

2.5 Ranuras de Expansión

Definición 2.5.1 Ranuras de expansión

Permiten la **conexión de otras tarjetas de circuito impreso a nuestra placa base** para añadir funcionalidades no incluidas. Por ejemplo, queremos un mejor controlador gráfico que el del procesador, así que conectamos una GPU (tarjeta gráfica) a través de esos conectores.

Definición 2.5.2 Riser card

Rotan 90° el conector para cuando tenemos un espacio limitado.

2.5.1. PCI y AGP

PCI y AGP

- **PCI** (*Peripheral Component Interconnect*)
 - **Plug and play**, puedo meter la tarjeta (no las programo) y negocio según lo que va bien en el momento. (puerto 3 39...)
 - El bus tiene una **señal de reloj** para sincronizar todas las señales
 - Half duplex igual que ddr. Puedo leer y escribir pero no de forma simultánea
 - El **ancho de banda** depende de la **frecuencia** de la señal del **reloj**. Frecuencia del reloj x número de bits que transfiero.
 - **Comparten bus** las diferentes ranuras de PCI. Eso significa que no pueden usar el bus a la vez.
- **AGP** (*Accelerated Graphics Port*)
 - PCI se quedó corto muy rápido (sobretudo para las GPU). Entonces Intel creó AGP solo para gráficas y solo tiene una ranura. Permite mejorar el estándar y conseguir un ancho de banda muy alto (2GBps).

Nota Distinguir entre PCI y PCIe

PCIe tienen una muesqucita chiquitita a la izquierda y además hay de varios tamaños. PCI tiene la muesqucita a la derecha. PCI es más alto que PCIe y PCIe permite usar tarjetas de tamaño diferente a la ranura.

2.5.2. PCI-Express (PCIe)

PCIe

- **Conexión serie:** a través de una línea voy enviando los bits sin depender del reloj.
- **Punto a punto:** es independiente, no compartes la información entre los dispositivos.
- **LANE**, comunicación: Cada lane son 4 cables, dos cables para recibir y dos para mandar. La comunicación es **full duplex**, puedo enviar y recibir al mismo tiempo.
- **Hot plug** es que puedo meter el dispositivo con el PC encendido, en caliente.
- La **transmisión** está **sincronizada** y el **reloj está embebido en los datos**. En este caso, usan una codificación que por esos dos cables puedes meter la información de envío y sincronización. Se llama **señal Manchester** y se crea a través de mezclar la señal del reloj con la de los datos.
- El envío de información, al ser **tipo serie**, puede tener **problemas de fiabilidad**. Para asegurar que hay pocos errores, decidieron que en las primeras versiones de PCI express se utilizara una codificación que, **por cada 10 bits** enviados, 8b son datos y **2 de redundancia**. 8b/10b. En las últimas versiones, es tan fiable que solo tenemos 2 bits de redundancia por cada 130bits enviados (128b/130b).
- Es **compatible hacia atrás**. Puedes conectar una GPU de versión mayor en una ranura de menor y al revés.
- Es un **estándar escalable**. Puedo multiplicar x3 o x6 el ancho de banda si lo necesito.

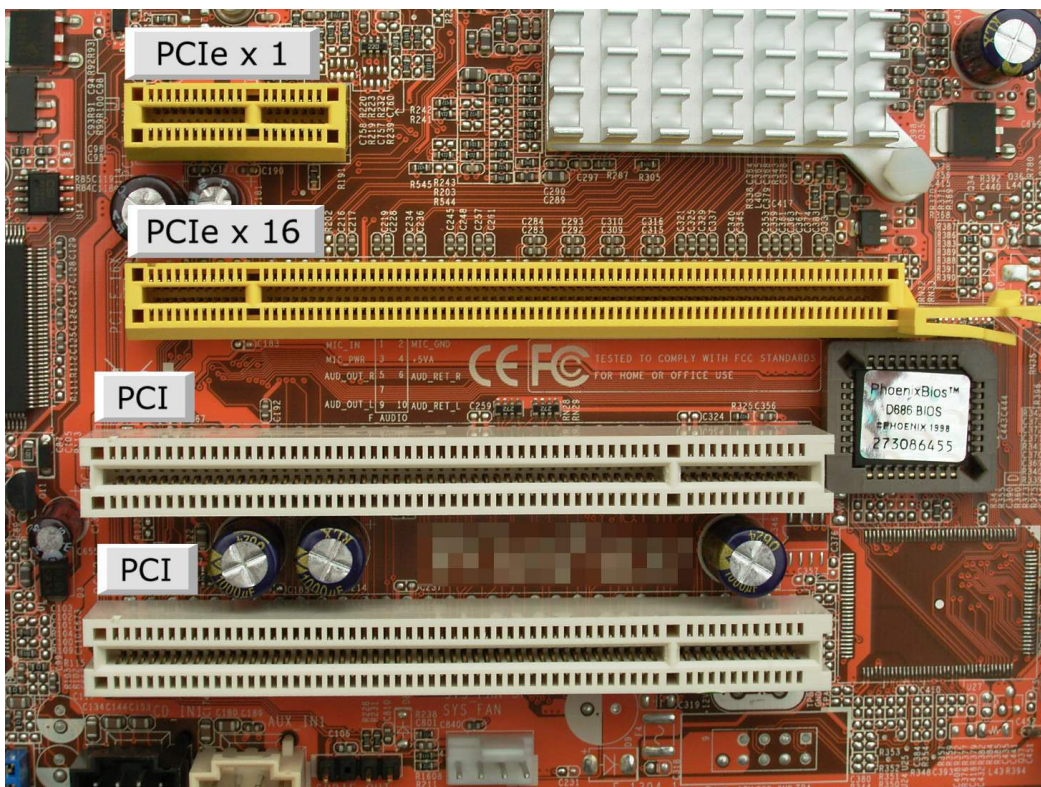


Figura 2.10: Comparativa PCI y PCIe

2.5.3. Ventajas de usar una interfaz serie vs paralela

- 1 **Mayor frecuencia de reloj** ya que evita el “desfase” entre las señales : Algunas señales tienen que llegar más lejos que otras en la placa base, así que se retrasan.
- 2 **Menor número de pistas** para un rendimiento similar (recordemos que pistas son los circuitos de cobre de la placa base).
- 3 **Mayor facilidad** para obtener conexiones full duplex. **No todas** las interfaces serie lo son.

2.6 Almacenamiento y E/S

2.6.1. Almacenamiento permanente

Conservan información incluso sin electricidad.

- Magnéticos: HDD (discos duros), cintas.
- Ópticos: CD, DVD, Blu-Ray.
- NVRAM: SSD.

2.6.2. Discos duros (HDD, *Hard Disk Drives*)

Características

- Tiene forma de disco, están cubiertos de material magnético.
- Magnetiza en una dirección u otra una zona en la que almacena información aunque se apague la electricidad.
- **No son de acceso aleatorio**, importa la dirección donde está el dato. La **latencia** (lo que se tarda en obtener el dato en específico) puede ser **enorme** porque se tiene que poner en la pista mecánicamente.
- Las escrituras no secuenciales tardan mucho.
- Los datos se distribuyen en **pistas** (*tracks*). Cada pista se subdivide en **sectores** de 512 bytes. Los sectores se agrupan en **clusters** lógicos.

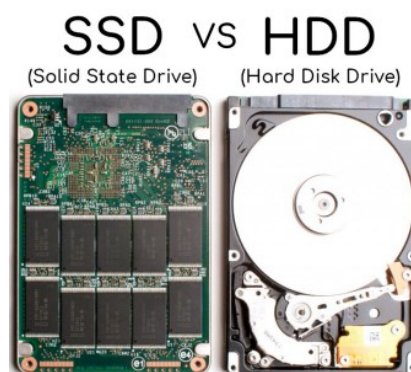


Figura 2.11: Comparativa entre SSD y HDD

2.6.3. Unidades de estado sólido (SSD, *Solid State Drives*)

Características

- Al ser chips, puedo **acceder** a la información de forma **paralela**. Además es **más rápido** con acceso aleatorio.
- En los servidores los SSD son más caros y tienen menos capacidad.
- Hay dos tipos de celda: **SLC** y **MLC**. En MLC uso 4 estados según la cantidad de electrones pero es muy poco fiable. Así, donde guardábamos 0 o 1 ahora guardamos 00 01 10 11.
- Si reescribo muchas veces se pierden más rápido las prestaciones. Hay un controlador en los chips que **nivela el desgaste** (*wear levelling*) de las celdas. Hay ssd que duran unos 5 o 6 años antes de dar fallos.
- Esta tecnología **no es óptima para el archivado** (guardar la información para el futuro). Los electrones se pueden escapar y se estropea la información. Los discos duros pueden fallar en la parte mecánica pero duran más.

2.6.4. RAID (*Redundant Array of Independent Disks*)

Definición 2.6.1 RAID

RAID (*Redundant Array of Independent Disks*) es un método de almacenamiento que **combina múltiples discos duros** en una sola entidad para proporcionar mejoras en el rendimiento, la capacidad o la redundancia. Existen dos tipos principales de RAID: hardware y software.

Definición 2.6.2 RAID hardware

Es un controlador RAID integrado en el disco duro o en una placa base que gestiona los discos duros conectados. Permite la gestión del RAID sin usar software adicional y es más rápido y confiable.

Definición 2.6.3 RAID software

Se ejecuta en el sistema operativo y permite la gestión de discos duros como un arreglo lógico que otorga las ventajas del RAID. No ofrece la misma velocidad de acceso al disco como el RAID hardware.

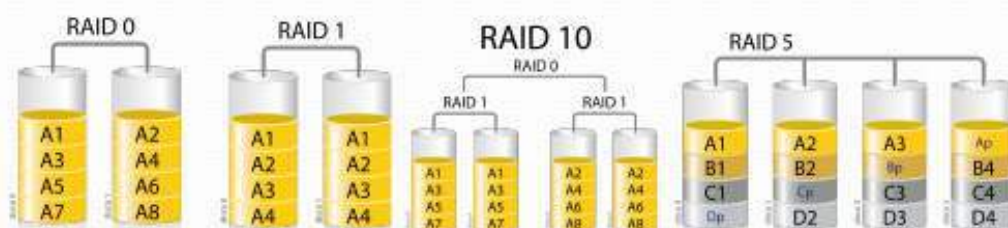


Figura 2.12: Diferentes tipos de RAID

Algunos tipos de RAID

- ❶ **RAID 0:** Divide los datos en fragmentos y los distribuye entre múltiples discos.
- ❷ **RAID 1:** Duplica los datos en dos o más discos; cada disco tiene una copia exacta.
- ❸ **RAID 1+0:** Combinación de RAID 1 y RAID 0. Los datos se dividen entre discos (RAID 0) y cada fragmento se duplica (RAID 1).
- ❹ **RAID 5:** Distribuye los datos y una "paridad" entre todos los discos. La paridad permite reconstruir datos si un disco falla.

2.6.5. Unidades ópticas

Almacenan la información de forma permanente (no volátil) a través de una serie de surcos en un disco que pueden ser leídos por un haz de luz láser. Los discos compactos (CD), discos versátiles digitales (DVD) y discos Blu-ray (BD) son los tipos de medios ópticos más comunes que pueden ser leídos y grabados por estas unidades.

2.6.6. Unidades de cinta

- Almacenan la información de forma **permanente** (no volátil) a través de una cinta recubierta de material magnético que se enrolla por medio de carretes.
- Las latencias suelen ser **muy altas** ya que hay que rebobinar la cinta hasta que el cabezal se encuentre en la posición deseada.
- Es el medio con la **mayor densidad de bits** para un área dada. Actualmente, permiten almacenamiento de decenas de TB por cinta y velocidades de lectura secuencial en torno a 150 MBps.
- Es el medio de almacenamiento masivo más barato.
- Se usan normalmente como almacenamiento de respaldo (**backup**) y archivado.

2.6.7. SATA

Definición 2.6.4 P-ATA

P-ATA, también llamado ATA, fue el predecesor de SATA. Es paralelo, half duplex y los dispositivos comparten el bus a pares.

Características

- Conexión serie.
- Half-duplex.
- Reloj embebido en datos.
- Codificación 8b/10b.
- Hot plug.
- Dos cables: uno de alimentación y otro de datos.
- **No se puede duplicar la frecuencia** como pasaba en PCIe porque exigiría mucha electricidad. **Limita la velocidad** del disco duro (600 mbps) y el **cable es muy corto**.

2.6.8. SCSI y SAS

SCSI

- Paralelo de 16b, Half duplex, hot-plug.
- Puedes conectar varias unidades en cadena (*daisy chain*). Ese disco duro tiene un conector de salida para otro duro etc etc. Un montón de discos duros conectados al mismo conector. No puedo acceder simultáneamente a ninguno de ellos.
- Ultra SCSI se puede usar para las unidades de cinta (que necesitaban solo 150MBps). Es paralelo.
- Es reemplazado por la versión serie de SCSI: SAS (*Serial Attached SCSI*)

SAS

- Serie punto a punto igual que SATA.
- Full duplex (leer y escribir simultáneamente en el disco)
- Hot plug (conectar en caliente)
- 8b/10b (igual que SATA, 2b de redundancia), el ancho de banda es el doble que SATA (1200mbs)
- Cable de hasta 10 metros.
- Compatible con discos SATA. Interfaz idéntica.

2.6.9. Conexión de múltiples unidades SAS

Para este propósito existen los conectores mini-SAS. Con un solo conector, puedes meter hasta 4 dispositivos (SAS o SATA). Y también están los Expansores SAS (SAS Expanders) que permiten múltiples conexiones SAS expandiendo la placa base. Ver 2.13.

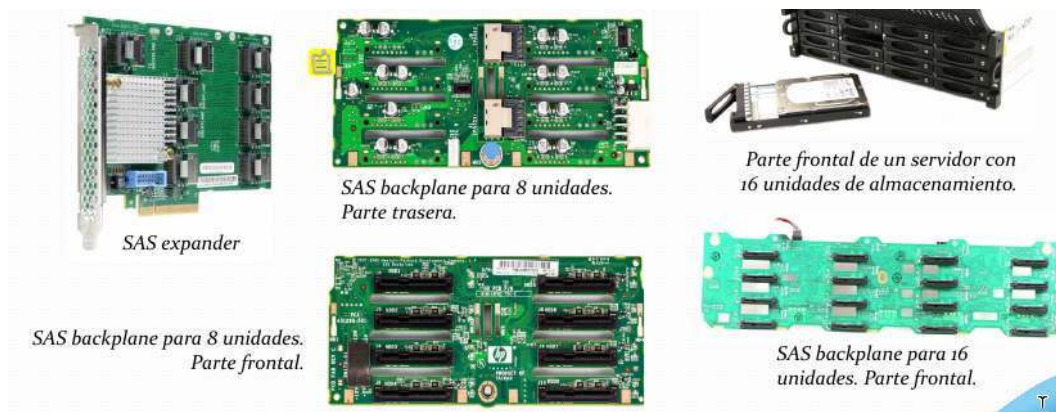


Figura 2.13: Expansores y backplane de SAS.

2.6.10. NVMe: Non-Volatile Memory Express

Permite el acceso a unidades de estado sólido (SSD) a través de PCIe. Utilizan PCIe x4 y permiten aprovechar al máximo la velocidad de los SSD. Se usan en portátiles porque ahorran espacio.



Figura 2.14: Protocolo NVMe, cómo se ve en las placas

2.7

Conectores del panel trasero

Nota Este backplane... ¿Es de PC o servidor?

Para saber si un backplane es de PC o servidor solo hay que revisar si tiene conectores de audio o de vídeo de altas prestaciones.

Definición 2.7.1 Ethernet

Ethernet un conjunto de tecnologías para interconectar equipos a través de red. Puede utilizar cables de par trenzado o fibra óptica (con adaptadores de señal).

Definición 2.7.2 USB

Interfaz de transmisión de datos serie, con reloj embebido, hot plug. En usb 3.0 (la mejora de 2.0) hay pines full duplex y se llegan hasta 10 veces más velocidad que el anterior (60MBps → 600MBps).

2.7.1. Conectores internos

La placa base puede tener conectores dentro opcionales. Todos tienen un **system pannel** (*front pannel*) que tiene la finalidad de conectar el chasis con la placa base. La mayoría de placas tienen un botón de reseteo de equipo. Puede tener varios leds que indican estados. Por ejemplo uno de "encendido" otro de "entrada salida en uso". También pueden tener un pequeño altavoz que suelte pitidos según códigos de error para informar al usuario (*speaker*).

Definición 2.7.3 ROM/Flash BIOS

Almacena el código de arranque (*boot*) del computador. Este código se encarga de identificar los dispositivos instalados, instalar drivers básicos para acceder a los mismos, realizar el Power-on self-test (POST) del sistema e iniciar el SO. Es una **memoria permanente** que aunque le quitemos la alimentación no va a perder su información. carga las instrucciones que tiene que ejecutar el núcleo principal para iniciar el sistema.

Definición 2.8.1 Chipset

El chipset es el conjunto de circuitos integrados (chips) de la placa base encargados de controlar la comunicación entre los diferentes componentes de la placa base. Un chipset se suele diseñar para una familia específica de microprocesadores.

Definición 2.8.2 Puente norte (*north bridge*)

Encargado de las transferencias de mayor velocidad (principalmente con el microprocesador, la memoria, la tarjeta gráfica y el puente sur).

Definición 2.8.3 Puente sur (*south bridge*)

Encargado de las transferencias entre el puente norte y el resto de periféricos con menores exigencias de velocidad de la placa.

Cuando se habla de chipset, normalmente, se habla de puente sur porque es el que queda. Hay pocas placas base que tengan aún puente norte.

Definición 2.8.4 System on a Chip (SoC)

Actualmente existe una configuración en la que no hay chipset porque el procesador lo hace todo (AMD EPYC). Tiene el controlador RAM, gráfico, ethernet, etc. en solo un procesador. En algunos ejemplos hay un chip adicional que se encarga de la administración remota del servidor y que debe realizarse aunque el PC esté apagado, pero el resto se hace en la CPU.

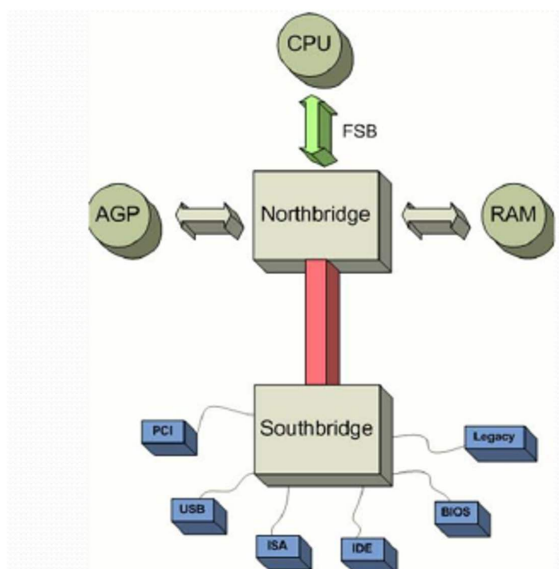


Figura 2.15: Esquema básico de un chipset

Tenemos dos formatos : **torre** y **rack**.

Definición 2.9.1 Rack

La unidad rack (U) se usa para describir la altura del equipamiento preparado para ser montado en un rack (estante) de 19 pulgadas de ancho (48,26 cm). Una unidad rack (1U) equivale a 1,75 pulgadas (4,445 cm) de alto.

La torre también tiene que tener un ancho estándar (basado en unidades rack [U]) por si queremos meterla en un armario.

Nota ¿Cómo distinguir una placa de servidor?

Tiene varias CPU, no tiene salida de vídeo, es grandota, puede tener conectores de fibra óptica o conectores mini-SAS. Puede tener grupos de ranuras de RAM separados, uno para cada CPU. Además, pueden tener formas más raras (alargadas, por ejemplo) para caber en boards con múltiples servidores.

Definición 2.9.2 Blade servers

Se trata de un gran armazón (*blade enclosure*) en el que se van añadiendo, a modo de "láminas" (*blades*) computadores "compactos" (*server blades*) que tienen el espacio estrictamente necesario para procesadores, módulos de DRAM y, según el modelo, un pequeño almacenamiento local. Este gran armazón proporciona a los server blades, entre otras cosas, alimentación, refrigeración y conectores externos para USB y Ethernet.

MONITORIZACIÓN DE SERVICIOS Y PROGRAMAS

3

3.1

Concepto de Monitor de Actividad

Definición 3.1.1 Carga (*workload*)

Conjunto de **tareas** que ha de realizar el servidor, es decir, todo aquello que **demande recursos** del servidor.

Definición 3.1.2 Actividad de un servidor

Conjunto de **operaciones** que se realizan en el servidor como **consecuencia de la carga** que soporta

Definición 3.1.3 Monitor de actividad

Cualquier herramienta diseñada para **medir la actividad** de un sistema informático y facilitar su análisis

¿Por qué monitorizar la actividad?

- Saber qué hardware no es suficiente para nuestros propósitos y cuál sustituir (**cuello de botella**).
- Predecir situaciones (momentos de mayor carga, *capacity planning*).
- Conocer partes críticas de nuestro código (*hot spots*).
- **Adaptar el SO** a la carga.

3.1.1. Clasificaciones de monitores de actividad

Según cómo se mide

- **Monitor por eventos:** Cada vez que ocurre algo (cambio en el sistema) da información exacta. Por ejemplo un archivo de log es un monitor por eventos.
- **Monitor por muestreo:** Cada T segundos (periodo de muestreo) el monitor mide. T puede variar. Otorga información estadística.
- Un monitor por **eventos no falla** pero si hay muchos eventos perdemos el control y en el monitor por **muestreo se controla la cantidad** de información que obtenemos.

Según cómo se mide

- **Software:** Programas instalados en el sistema. Cuando se ejecutan usan mi ram, mi cpu, mi almacenamiento, etc. **La sobrecarga es alta.**
- **Hardware:** Dispositivos físicos de medida. **No hacen sobrecarga**, muestran temperatura de partes del chasis, códigos de error de la placa, reloj de la cpu, etc.

Según interacción con el admin

- **No interacciona:** La consulta sobre los resultados se realiza aparte mediante otra herramienta independiente al monitor (**segundo plano, batch monitors**).
- **Si interacciona:** Se pueden consultar los valores mientras monitorizas e interactuar con los datos (con gráficos, modificando parámetros) (**primer plano, on-line monitors**).

3.1.2. Atributos de un monitor

Definición 3.1.4 Anchura de entrada

Cuánta información se almacena (recursos consumidos) por cada medida que toma el monitor

Definición 3.1.5 Sobrecarga (*Overhead*)

Qué recursos le “roba” el monitor al sistema

Fórmula 3.1.1 Sobrecarga

Al usar mi monitor imagina que necesitaran 10 Bytes y tengo 100 Bytes de memoria, pues $10/100 = 10\%$ de mis recursos se gastan (sobrecarga) de mi servidor.

$$Sobrecarga_{Recurso}(\%) = \frac{Recurso\ monitor}{Recurso\ sistema} \times 100$$

$$Sobrecarga_{Recurso}(\%) = \frac{Uso\ del\ recurso\ por\ parte\ del\ monitor}{Capacidad\ total\ del\ recurso} \times 100$$

$$Sobrecarga_{Recurso}(\%) = \frac{Tiempo\ de\ ejecucin\ del\ monitor}{Intervalo\ de\ medida} \times 100$$

Un ejemplo: Sobrecarga de CPU, el monitor se activa cada 5 segundos, gasta 6ms de la CPU.

$$Sobrecarga_{CPU}(\%) = \frac{6 \times 10^{-3}s}{5s} \times 100 = 0,12\%$$

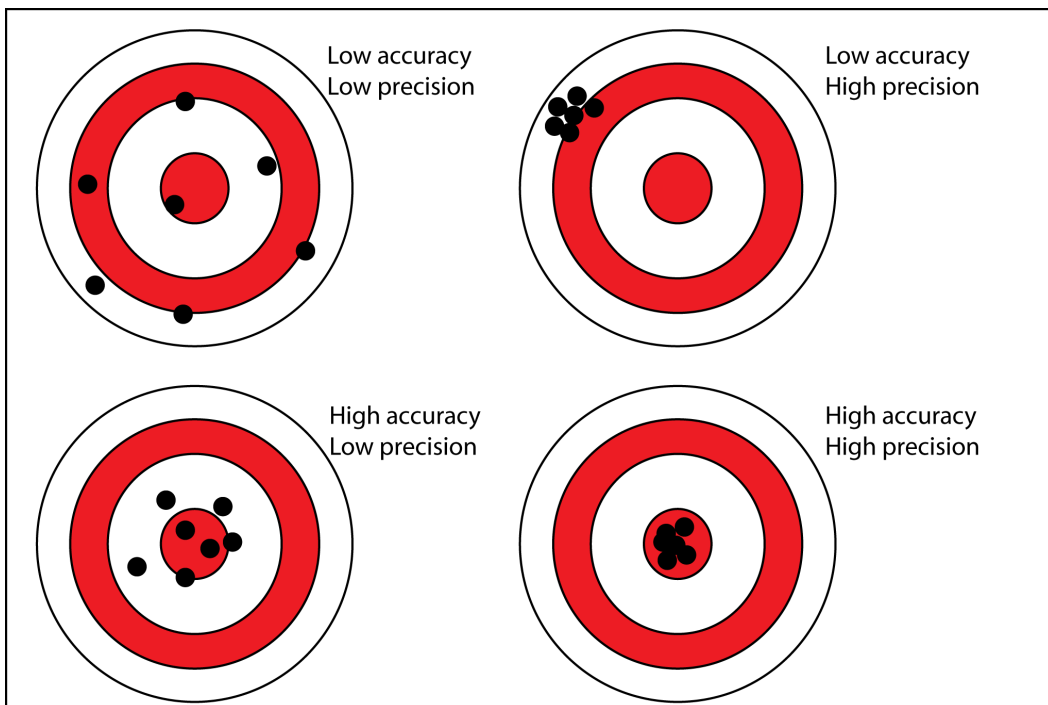


Figura 3.1: Precisión vs Exactitud (*accuracy*)

Definición 3.1.6 Exactitud del sensor(*Accuracy, offset*)

Lo que mido vs lo que debería estar midiendo de verdad. Mi reloj vs el reloj atómico del satélite. Si está descalibrado es un offset (**diferencia**) entre el valor real y mi reloj.

Definición 3.1.7 Precisión del sensor

A veces se toma una medida varias veces y hay **diferencias entre ellas**.

Definición 3.1.8 Resolución del sensor

Cuánto tiene que cambiar el valor a medir para detectar un cambio

3.2.1. Directorio `/proc` (Linux)

Es un directorio virtual utilizado por el núcleo de Linux para facilitar el acces del administrador a las estructuras de datos del S.O. A través de `/proc` podemos:

- Acceder a información global sobre el S.O
- Acceder a información de cada uno de los procesos del sistema (`/proc/[pid]`)
- Acceder y, a veces, modificar algunos parámetros del kernel del S.O (`/proc/sys`)

En Linux, la mayoría de los monitores de actividad a nivel de sistema usan como fuente de información este directorio

Comando 3.2.1 `uptime`

Tiempo que lleva el sistema en marcha y la “carga media” que soporta.

3.2.2. Carga del sistema, según Linux

Hay 3 tipos de estados básicos de un proceso:

- 1 En **Ejecución** (*running*) o esperando que haya un núcleo de CPU libre para poder ser ejecutado (*runnable*). La cola de procesos (*run queue*) está formada por aquellos que se están ejecutando y los que pueden ejecutarse (*runnable + running*).
- 2 **Bloqueado** esperando a que se complete una operación de E/S para continuar (*uninterruptible sleep = I/O blocked*).
- 3 **Durmiendo** esperando a un evento del usuario o similar que lo despierte (p.ej. una pulsación de tecla) (*interruptible sleep*).

Definición 3.2.1 Carga del sistema (Linux)

Número de procesos en **ejecución o bloqueados**.

Comando 3.2.2 `ps` (*process status*)

Muestra información sobre el **estado actual de los procesos** del sistema. Cuenta el 100 % de los recursos relativo a tus recursos en la máquina. Si tienes, por ejemplo, 4 CPUs, entonces 4 será 400 %. `ps` tiene **mucho overhead** y hay que **llamarlo cada cierto tiempo**, así que no nos ayuda mucho en nuestro entorno.

```
Procesos a mostrar:
-A, -e: show all processes; T: all processes on this
terminal; U: processes for specified users...
Campos que mostrar: process ID, cumulative user time,
number of minor/major page faults, parent process ID,
RSS, time process was started, user ID number, user
name, total VM size in bytes, kernel scheduling
priority, etc.

$ ps aur
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
miguel    29951 55.9   0.1 1448   384 pts/0    R   09:16 0:11 tetris
carlos    29968 50.6   0.1 1448   384 pts/0    R   09:32 0:05 tetris
javier    30023  0.0   0.5 2464  1492 pts/0    R   09:27 0:00 ps aur
```

Figura 3.2: Comando ps y su salida

Comando 3.2.3 top

Muestra **cada T segundos** valores globales de carga, procesos, utilización de CPU, uso de DRAM y SWAP, y valores particulares de cada proceso. Tiene **mucho overhead** porque es en **primer plano** y cada cierto tiempo me lo actualiza (también es **interactivo**).

algunos valores importantes a saber del comando top

- US: Tiempo ejecutando procesos del usuario a los que no se les ha cambiado la prioridad
- SY: Tiempo ejecutando procesos del kernel.
- ID: Idle, no hay nada que hacer.
- WA: No hay nada que hacer PORQUE estoy esperando entrada/salida y si fuera más rápida seguro estaría trabajando.

```
8:48am up 70 days, 21:36, 1 user, load average: 0.28, 0.06, 0.02
Tareas: 47 total, 2 running, 45 sleeping, 0 stopped, 0 zombie
%Cpu(s): 99.6 us, 0.3 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 256464 total, 234008 used, 22456 free, 13784 buffers
KiB Swap: 136512 total, 4356 used, 132156 free, 5240 cached Mem

  PID USER      PR  NI  VIRT  RES  SHR  STAT  %CPU  %MEM  TIME  COMMAND
9826 carlos    0   0   388   388   308  R    99.6   0.1  0:22  simulador
9831 miguel    19   0   976   976   776  R     0.3   0.3  0:00  top
   1 root       20   0    76    64    44  S     0.0   0.0  0:03  init
   2 root       20   0     0     0     0  S     0.0   0.0  0:00  keventd
   4 root       20  19     0     0     0  SN    0.0   0.0  0:00  ksoftiq
```

Figura 3.3: Comando top y su salida

Comando 3.2.4 `vmstat` (*virtual memory statistics*)

Mientras que `top` es puntual pero a nivel global de recursos y consume demasiado, `vmstat` resuelve esos problemas. Da información **cada cierto tiempo** sobre procesos que están **running**, **idle** o **blocked**. Tiene un fichero en `/proc` y un **overhead mínimo**. La primera línea es info desde el inicio del sistema y **no nos sirve** para nada.

Algunos elementos de `vmstat`

- `procs` (procesos):
 - `r`: running/runnable
 - `b`: Bloqueados por E/S
- `io` (e/s):
 - `bi`: blocks in
 - `bo`: blocks out
- `swap` (memoria virtual):
 - `si`: swapped in
 - `so`: swapped out
- `system` (sistema en general):
 - `in`: interrupciones por seg.
 - `cs`: cambios de contexto por seg.

% vmstat 1 6																
procs		memory				swap		io		system		cpu				
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
0	0	0	868	8964	60140	342748	0	0	0	23	7	222	199	1	4	80
0	0	0	868	8964	60140	342748	0	0	0	14	283	278	0	7	80	23
0	0	0	868	8964	60140	342748	0	0	0	0	218	212	6	2	93	0
0	0	0	868	8964	60140	342748	0	0	0	0	175	166	3	3	94	0
0	0	0	868	8964	60140	342752	0	0	0	2	182	196	0	7	88	5
0	0	0	868	8968	60140	342748	0	0	0	18	168	175	3	8	69	20

Figura 3.4: Comando `vmstat` y su salida

3.2.3. El monitor `sar` (*system activity reporter*)

Recopila información sobre la **actividad del sistema actual** (interactivo) y **histórica** (en `/var/log/sysstat/saDD` donde `DD` indica día del mes, pero puedes cambiarlo). `sar` es el frontend, mientras que `sadc` es el que a `proc` y mide la información.

Información de `sar` so- bre discos

- `tps`: transferencias por segundo.
- `rd_sec/s` y `wr_sec/s`: lecturas por segundo y escrituras por segundo.
- `rq`: bloques que pide
- `await`: tiempo de respuesta del disco. Toma en cuenta tiempo de cola y servicio.
- `svctm`: tiempo de servicio una vez superada la cola.
- `util`: porcentaje de utilización. No es capacidad de almacenamiento, sino el ancho de banda del sata para utilizar el disco.
- `-b`: entrada/salida
- `-d`: discos

```

Modo interactivo: [tiempo_muestreo, [nº muestras]]

Modo no interactivo:
-f Fichero de donde extraer la información, por defecto: hoy
-s Hora de comienzo de la monitorización
-e Hora de fin de la monitorización

-u Utilización global del procesador (opción por defecto)
-P Mostrar estadísticas por cada núcleo (-P ALL: todos)
-I Estadísticas sobre interrupciones
-W Cambios de contexto
-q Tamaño de la cola y carga media del sistema
-b Estadísticas globales de transferencias de E/S
-d Transferencias para cada disco
-n Conexión de red
-r Utilización de memoria
-R Estadísticas sobre la memoria
-A Toda la información disponible
...

```

Figura 3.5: Comando sar

```

$ sar (=sar -u)
00:00:00 CPU %usr %nice %sys %wa %st %idle
00:05:00 all 0.09 0.00 0.08 0.00 0.00 99.83
00:10:00 all 0.01 0.00 0.01 0.00 0.00 99.98
00:15:00 ...

```

Figura 3.6: Por defecto me da la información recopilada de hoy, (me da lo mismo que vmstat) a partir de las 0h de ese día.

```

$ sar -P ALL 1
19:30:39 CPU %usr %nice %sys %wa %st %idle
19:30:40 all 53.45 0.00 6.18 0.00 0.00 40.37
19:30:40 0 49.49 0.00 5.05 0.00 0.00 45.45
19:30:40 1 51.61 0.00 6.45 0.00 0.00 41.94
19:30:40 2 58.16 0.00 8.17 0.00 0.00 33.67
19:30:40 3 54.55 0.00 5.05 0.00 0.00 40.40
19:30:40 CPU %usr %nice %sys %wa %st %idle
19:30:41 all 50.49 0.00 6.19 0.00 0.00 43.32
...

```

Figura 3.7: -P de cada CPU, si pongo 1 de la primera y ALL son todos. Lo desglosa por procesador y el global. Con el número indica el intervalo de muestreo (modo **interactivo**) y cada segundo mide la información.

```
$ sar -b -s 10:00:00 -e 12:00:00 -f /var/log/sysstat/sa06
10:00:00 tps rtps wtps bread/s bwrtn/s
10:05:00 0.74 0.39 0.35 7.96 3.27
10:10:00 65.12 59.96 5.16 631.62 162.64
10:15:00 ...
```

```
$ sar -d 10 2
18:46:09 DEV tps rd_sec/s wr_sec/s avgrq-sz avgqu-sz await svctm %util
18:46:19 sda 1.70 33.60 0.00 19.76 0.00 0.47 0.47 0.08
18:46:19 sr0 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
18:46:19 DEV tps rd_sec/s wr_sec/s avgrq-sz avgqu-sz await svctm %util
18:46:29 sda 8.60 114.40 518.10 73.55 0.06 7.12 0.93 0.80
18:46:29 sr0 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
```

Figura 3.8: Ejemplo, pido el día 6 del mes mas allá de las 10, el de abajo pido cada 10 segundos 2 muestras.

Monitorizar red

```
$ sar -n TCP,ETCP,DEV 1
Linux 3.2.55 (test-e4fla80b) 08/18/2014 _x86_64_ (8 CPU)

09:10:43 PM IFACE rxpck/s txpck/s rxkB/s txkB/s rxcmp/s txcmp/s rxmst/s
09:10:44 PM lo 14.00 14.00 1.34 1.34 0.00 0.00 0.00
09:10:44 PM eth0 4114.00 4186.00 4537.46 28513.24 0.00 0.00 0.00

09:10:43 PM active/s passive/s iseg/s oseg/s
09:10:44 PM 21.00 4.00 4107.00 22511.00

09:10:43 PM atptf/s estres/s retrans/s isegerr/s orsts/s
09:10:44 PM 0.00 0.00 36.00 0.00 1.00
[...]
```

Figura 3.9: -n es la conexión de red. Para este monitor la e/s la desglosa en unidades de almacenamiento y red. En otros casos podríamos juntar lo pero para este monitor no. En este ejemplo, se ve cada segundo TCP y sus errores

Almacenamiento de datos del data collector (sadc)

Se utiliza un fichero de datos por día y se programa la ejecución de `sadc` x veces por cada fichero con `cron`. Además se añade un registro binario con los datos al histórico diario. Ver 3.10

```
%ls /var/log/sysstat
-rw-r--r-- 1 root root 3049952 Oct 1 23:55 sa01
-rw-r--r-- 1 root root 3049952 Oct 2 23:55 sa02
-rw-r--r-- 1 root root 3049952 Oct 3 23:55 sa03
-rw-r--r-- 1 root root 3049952 Oct 4 23:55 sa04
-rw-r--r-- 1 root root 3049952 Oct 5 23:55 sa05
-rw-r--r-- 1 root root 3049952 Oct 6 23:55 sa06
-rw-r--r-- 1 root root 3049952 Oct 7 23:55 sa07
-rw-r--r-- 1 root root 2372320 Oct 8 18:45 sa08
```

Día actual

Figura 3.10: En este ejemplo guarda información cada 5 minutos (por eso las ediciones de los días anteriores acaban a las 23:55) y el día actual del mes es 8 porque aún no escribió el último registro (el último es de las 18:45)

Nota Calcular la anchura de entrada (carga) del monitor

Primero miramos el archivo de `/var/log/sysstat` con `ls`. Lo que más nos interesa es cuántos bytes tiene y cuándo fue la última modificación. Tenemos que pensar: Si la última muestra fue, por ejemplo, a las 23:58, ¿cada cuánto actualiza `sar`? Pues 23:58 es 1438 minutos, ¿por qué número es divisible eso? Por 2 y por 719 (y por 2×719). Pues, si asumimos que se ejecuta cada 2 minutos, solo tenemos que dividir el tiempo del día entre 2 para saber cuántas muestras tomaríamos. Por nuestra descomposición de antes sabemos que $1438/2$ es 719, así que esas son las muestras que tomamos. ¿Cuál es la carga? **Peso del archivo/num muestras**. Si pesara 3MB, pues $3\text{MB}/719$ muestras. Si solo tomamos una muestra, pues la carga serían $3/1 = 3$ MB.

3.2.4. Método USE (Utilization, Saturation, Errors)

Para cada recurso comprobaremos, según un intervalo:

- ❶ **Utilización media:** tiempo de CPU (con `sar -P`, por ejemplo), memoria ocupada (`vmstat`), ancho de banda del disco...
- ❷ **Saturación:** Ocupación media de las colas (por ejemplo, `sar -d` con `avgqu-sz`)
- ❸ **Errores:** Mensajes de error de los recursos (por ejemplo, `dmesg`)

3.2.5. Herramientas de monitorización de servidores distribuidos

Normalmente si tienes varios servidores controlados desde el ordenador del administrador, buscas programas que tengan **interfaz gráfica** como Elastic, Zabbix o Grafana.

Características

- **Escalables:** Puedes tener **mini monitores** encargados de subconjuntos de servers. También puedes tener **servidores independientes** para base de datos y frontend y **descubrir automáticamente** nuevos servers.
- **Extensibilidad:** Plugins para más funcionalidad (**scripts**), añades otros protocolos (como SNMP), añades API para aplicaciones (como Grafana).
- **Prestaciones:** permitir *polling* (buscar constantemente información) o *pushing* (mandar información automáticamente) con o sin agente. Envías por lotes la información o la comprimes para no quedarte sin espacio.
- **Fiabilidad:** Validar valores que recibes.
 - **+ disponibilidad:** te mandan alarmas y notificaciones.
- **Seguridad:** Encriptar comunicación y evitar suplantación con hashing.

Definición 3.3.1 Profiler

Monitorizar la actividad generada por una aplicación concreta con el fin de **obtener información para poder optimizar su código**.

Definición 3.3.2 Hot spot

Parte del código donde está la mayoría del tiempo de ejecución.

3.3.1. `usr/bin/time`

El programa `usr/bin/time` mide el tiempo de ejecución de un programa y muestra algunas estadísticas sobre su ejecución. **(NO ES UN PROFILER AÚN)** Es diferente a `time`, hay que poner específicamente el directorio.

Estadísticas

- User time: tiempo de CPU ejecutando en modo usuario.
- System time: tiempo de CPU ejecutando código del núcleo del S.O.
- Elapsed (wall clock) time: tiempo que ha pasado desde el comienzo hasta la finalización del programa.
- Major page faults: fallos de página que requieren acceder al almacenamiento permanente.
- Cambios de contexto voluntarios: Cuando acaba el programa o al tener que esperar a una operación de E/S cede la CPU a otro proceso.
- Cambios involuntarios: Expira su "time slice".

3.3.2. Profiler `gprof`

Estima el tiempo de CPU que consume cada función de un proceso/hilo. También calcula el número de veces que se ejecuta cada función y cuántas veces una función llama a otra. Utilización de `gprof` para programas escritos en C, C++:

Funcionamiento

- 1 Arranque:
 - 1 Genera **tabla con dirección en memoria** de cada función
 - 2 **Contadores** de cada función de programa a **0** (veces que se ejecuta y estimar tiempo en CPU)
 - 3 El SO pone un **temporizador**.
- 2 Ejecución:
 - 1 Cada vez que **ejecuta una función**: Subir numero de ejecuciones y mira qué función la ha llamado.
 - 2 Cada vez que el **temporizador llega a 0 segundos**, se interrumpe el programa y se suma el contador de función interrumpida. Reinicia el contador.
- 3 Final:
 - 1 Teniendo en cuenta el tiempo total de CPU y los contadores de CPU, estima el CPU de cada función.
 - 2 Genera *flat profile* (cuál función gasta más tiempo) y *call profile* (cuál función gasta más tiempo de llamadas de sistema).

flat profile

- %time: % del tiempo total de CPU del programa que usa código propio de la función (no las llamadas a otras funciones)
- cumulative seconds: La suma acumulada de los segundos consumidos (CPU) por el código propio de la función y por el de las funciones que aparecen encima de ella en la tabla.
- self seconds: tiempo (CPU) total de ejecución del código propio de la función.
- self s/call: tiempo (CPU) medio de ejecución del código PROPIO por cada llamada a la función.
- total s/call: tiempo (CPU) medio de ejecución de cada llamada a la función (propio + llamadas).

call profile

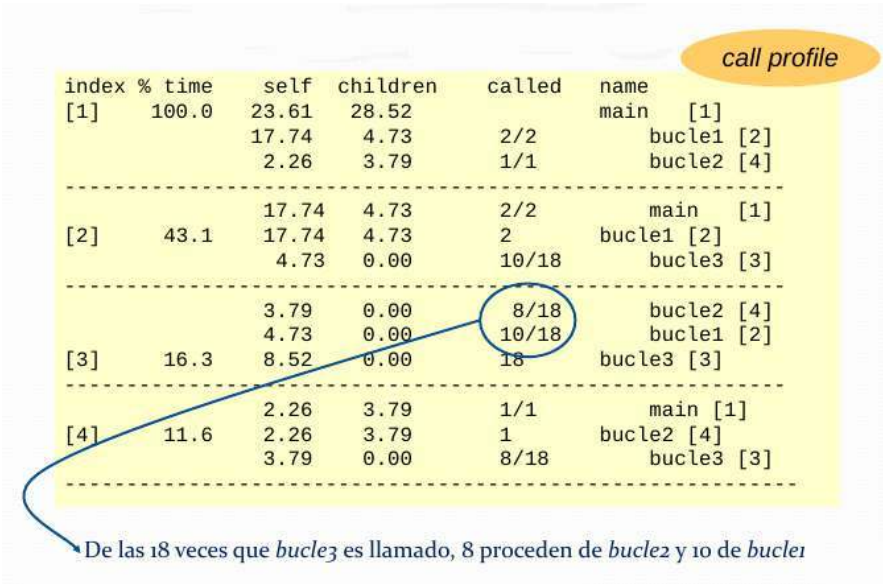


Figura 3.11: Explicación de *call profile* con un ejemplo.

3.3.3. perf

Es un conjunto de herramientas para el análisis de rendimiento en Linux, basadas en eventos software y hardware (hacen uso de contadores hardware disponibles en los últimos microprocesadores de Intel y AMD). Permiten analizar el rendimiento de a) un hilo individual, b) un proceso + sus hijos, c) todos los procesos que se ejecutan en una CPU concreta, d) todos los procesos de un determinado usuario (o de todos) que se ejecutan en el sistema.

Algunos comandos

```
usage: perf [-version] [-help] COMMAND [ARGS]
  • list: Lista todos los eventos disponibles
  • stat: Cuenta el número de eventos.
  • record: Recolecta muestras cada vez que se produce un determinado conjunto de eventos. Fichero de salida: perf.data.
  • report: Analiza perf.data y muestra las estadísticas generales.
  • annotate: Analiza perf.data y muestra los resultados a nivel de código ensamblador y código fuente (si está disponible).
```

3.3.4. valgrind

valgrind es un conjunto de herramientas para el análisis y mejora del código de un programa (sólo disponible en Linux). Entre éstas, encontramos herramientas para la detección de errores de memoria (*memcheck*), detección de bloqueos o carreras en hebras (*hellgrind*), análisis del uso del heap (*massif*) y un profiler de memoria caché y de fallos de predicción de saltos (*cachegrind*).

valgrind analiza cualquier programa ya compilado (no necesita instrumentar el programa a partir de su código fuente).

valgrind actúa, esencialmente, como una máquina virtual que emula la ejecución de un programa ejecutable en un entorno aislado. Por tanto, la información suministrada es exacta y no estadística.

Como desventaja, el sobre coste computacional es muy alto. La emulación del programa ejecutable puede tardar decenas de veces más que la ejecución directa del programa de forma nativa.

3.3.5. v-tune (Intel) y CodeXL (AMD)

Al igual que perf, pueden hacer uso tanto de eventos software como hardware. Ambos programas funcionan tanto para Windows como para Linux, y permiten obtener información sobre los fallos de caché, fallos de TLB, bloqueos/rupturas del cauce, fallos en la predicción de saltos, cerrojos y esperas entre hebras, etc. asociados a cada línea del programa (tanto en código fuente como en código ensamblador).

También son capaces de medir el rendimiento de GPU, controlador de memoria, conexiones internas a la CPU, etc. y permiten la ejecución remota.

ANÁLISIS COMPARATIVO DEL RENDIMIENTO

4.1

Referenciación (*benchmarking*)

Características de un buen índice de rendimiento

- **Repetibilidad:** Siempre que se mida el índice en las mismas condiciones, el valor de éste debe ser el mismo.
- **Representatividad y fiabilidad:** Si un sistema A siempre presenta un índice de rendimiento mejor que el sistema B, es porque siempre el rendimiento real de A es mejor que el de B.
- **Consistencia:** El índice se debe poder medir en cualquier sistema informático, independientemente de su arquitectura o de su S.O.
- **Facilidad de medición:** Sería deseable que las medidas necesarias para obtener el índice sean fáciles de tomar.
- **Linealidad:** Sería deseable que, si el índice de rendimiento aumenta, el rendimiento real del sistema aumente en la misma proporción.

4.1.1. Búsqueda de un buen índice de rendimiento para un componente concreto: CPU

El tiempo de ejecución, la frecuencia de reloj y CPI no nos sirven porque hay sistemas con frecuencia peor que tienen mejores prestaciones. Las CPI y el tiempo de ejecución dependen del programa que se ejecute.

Los MIPS varían incluso en diferentes programas con el mismo ordenador y depende del juego de instrucciones del procesador (RISC, CISC)

El formato de números en coma flotante varía entre arquitecturas y no representan una medida relevante para los servidores (potencialmente)

La elección final es el tiempo de ejecución teniendo en cuenta que **dependerá de la carga que se use para la comparación.**

4.1.2. Carga real

Elegir la carga para comparar servidores es difícil porque:

- ❶ Varía con el tiempo.
- ❷ Es difícil reproducirla.
- ❸ Puede verse afectada por la velocidad del servidor.

Por eso usamos un modelo de carga (*test workload*). Este modelo tiene que ser **representativo de la carga real** y lo más **simple** posible.

Estrategias para modelar los de carga

- **Personalizado** a partir de monitorización del sistema (caracterización de carga).
- **Genérico**: Cargas de prueba similares a lo que se quiere reproducir (referenciación o *benchmarking*)

Caracterización de la carga

Pasos

- 1 Elegimos **recursos** de la carga (CPU, RAM, disco...)
- 2 **Parámetros** de los recursos (lectura/escritura en RAM, ciclos de CPU...)
- 3 **Medir** valor de parámetros con monitores (**muestreo**)
- 4 **Analizar** datos
- 5 **Generar modelo** de carga seleccionando representantes + info estadística del tiempo.

Es un modelo muy **representativo** de la carga real, pero es bastante **tedioso** y **no conocemos el rendimiento de otros servidores** con nuestra carga.

Referenciación (*benchmarking*)

Definición 4.1.1 *Benchmark Program*

Programa estándar para comparar rendimiento entre sistemas informáticos

Características

- Carga de prueba específica.
- Conjunto de reglas para la correcta ejecución, obtención y validación de los resultados.
- Índice de rendimiento (métrica) para realizar comparaciones.

Ventajas

- Probable encontrar uno que reproduzca **condiciones parecidas** a las de nuestro servidor.
- **Comparaciones de rendimiento justas**, porque siguen las reglas del benchmark.
- Muchos permiten **ajustar la carga** para medir la escalabilidad de nuestro servidor.
- Conocemos tanto el rendimiento para un benchmark de diferentes servidores como su diseño y configuración, obteniendo **info para mejorar** los nuestros.

4.1.3. Clasificación benchmark

Según la estrategia de medida

- Programas que miden el tiempo necesario para ejecutar una cantidad pre- establecida de tareas: La mayoría de benchmarks.
- Programas que miden la cantidad de tareas ejecutadas para un tiempo de cómputo preestablecido:
 - **SLALOM**: Mide la exactitud de la solución de un determinado problema que se puede alcanzar en 1 minuto de ejecución.
 - **TPC-C**: Calcula cuántas consultas por segundo se realizan, de media, a un servidor de base de datos permitiendo aumentar el nº de usuarios. Exige un tiempo mínimo de respuesta para un determinado tanto por ciento de peticiones.

- **Microbenchmarks** o benchmarks para componentes: estresan componentes o agrupaciones de componentes concretos del sistema: procesador, caché, memoria, discos, red, procesador+caché, procesador+compilador+memoria virtual, etc.
- **Macrobenchmarks** o benchmarks de sistema completo o de aplicación real: la carga intenta imitar situaciones reales (normalmente servidores con muchos clientes) típicas de algún área. P.ej. comercio electrónico, servidores web, servidores de ficheros, servidores de bases de datos, sistemas de ayuda a la decisión, paquetes ofimáticos + correo electrónico + navegación, etc.

Ejemplos de Microbenchmarks

- Whetstone: Rendimiento de operaciones en coma flotante con aplicaciones científicas de sumas, multiplicaciones y funciones.
- Linpark : Rendimiento de operaciones en coma flotante con un algoritmo para resolver ecuaciones lineales.
- Dhrystone: Rendimiento de operaciones con enteros con copia y comparación de cadenas de caracteres.
- Stream: Ancho de banda de DRAM y SRAM.
- IOzone: Rendimiento de sistema de ficheros. También HD Tune, fio o hddparm.
- Iperf : Rendimiento TCP y UDP (cliente y servidor). Pchar, ancho de banda por salto IP.
- Paquetes de microbenchmarks:
 - AIDA64
 - Sandra
 - Phoronix Test Suite: open source, genera informes y test de rendimientos automáticamente. Recopila info de sensores del sistema en ejecución, permite visualizar el resultado de forma gráfica y exportarlo.

4.1.4. El paquete de microbenchmarks SPEC CPU

Es un **conjunto de pruebas de rendimiento para procesadores** diseñadas por el *Standard Performance Evaluation Corporation* (SPEC). Se utiliza para **evaluar la velocidad y eficiencia** energética de los procesadores. Se distribuye como **imagen ISO** con el código, datasets, herramientas y documentación. La componen 4 conjuntos de microbenchmarks:

- SPECspeed@2017 Integer (**tiempo de respuesta** en aritmética entera)
- SPECspeed@2017 Floating Point (**tiempo de respuesta** en coma flotante)
- SPECrate@2017 Integer (**productividad** (cuántos programas/tiempo) en aritmética entera)
- SPECrate@2017 Floating Point (**productividad** en coma flotante)

Utiliza **programas reales** que tengan **gran portabilidad** entre arquitecturas (gcc, xz, lbm...).

Índice de prestaciones de SPECspeed

Hay dos tipos:

- ❶ Base: Todos los programas escritos en el mismo lenguaje usan las mismas opciones de compilación.
- ❷ Peak: Permite que cada uno escoja las opciones de compilación óptimas.

Cada programa se ejecuta 3 veces y se escoge le medio. El **índice SPEC** es la media geométrica de las **ganancias en velocidad con respecto a una máquina de referencia**.

Fórmula 4.1.1 Índice de prestaciones de SPEC

$$\sqrt[n]{\frac{t_1^{REF}}{t_1} \times \frac{t_2^{REF}}{t_2} \times \dots \times \frac{t_n^{REF}}{t_n}}$$

4.1.5. Benchmarks de sistema completo: TPC

Definición 4.1.2 TPC (*Transactions Processing Performance Council*)

Organización sin ánimo de lucro especializada en benchmarks relacionados con el procesamiento de grandes cantidades de datos.

Principales tipos de benchmark que ofrece TCP

- **OLTP** (*on-line transaction processing*): Se trata de realizar pequeñas transacciones (compras, consultas,...) con unos determinados requisitos de tiempos de respuesta.
- **DS** (*decision support*): Se trata de realizar consultas complejas que suelen requerir acceso a buena parte de una gran base de datos y un procesamiento posterior.
- **IoT** (*Internet of Things*): Se trata de procesar una gran cantidad de datos procedentes de una gran cantidad de dispositivos diferentes.
- **Big Data**: Se trata de procesar una gran cantidad de información heterogénea usando clusters de computadores y Apache Hadoop.
- **Artificial Intelligence**: Se trata de ejecutar algoritmos de aprendizaje utilizados en *machine learning* (p.ej. redes neuronales profundas) a partir de datos.
- **Virtualization**: Se trata de ejecutar los benchmarks de los tipos anteriores en un entorno virtualizado en lugar de en una máquina "física" (*bare metal*).

Ejemplos: Benchmark TPC-C y TPC-H

TPC-C es de tipo OLTP y simula una gran compañía. Las peticiones involucran acceso a las bases de datos tanto locales como distribuidas, ejecución simultánea de consultas y acceso no uniforme a las bases de datos.

TPC-H es de tipo DS, examinar grandes volúmenes de datos para poder contestar a preguntas complejas. Aunque es escalable, la base de datos cuenta con la información de un mínimo de 10000 proveedores con un mínimo de 10 millones de filas.

4.2

Análisis de los resultados de un test de rendimiento

Muchos tests de rendimiento se basan en la ejecución de diferentes programas y, por tanto, producen diferentes medidas de rendimiento. Sin embargo, estos tests suelen resumir todas estas medidas en un **único** valor: el índice de rendimiento de dicho test.

Método habitual de síntesis: uso de algún tipo de **media**.

Fórmula 4.2.1 Media Aritmética

Dado un conjunto de n medidas $t_1, \dots, t_n$, definimos su media aritmética:

$$\bar{t} = \frac{1}{n} \sum_{k=1}^n t_k$$

Si no todas las medidas tienen la misma importancia, se puede asociar a cada medida t_k un peso w_k , obteniéndose la media aritmética ponderada.

$$\bar{t}_w = \frac{1}{n} \sum_{k=1}^n w_k \times t_k \text{ con } \sum_{k=1}^n w_k = 1$$

Si t_k es el tiempo de ejecución del programa de benchmark k -ésimo en la máquina a testar, w_k podría escogerse, por ejemplo, inversamente proporcional a dicho tiempo de ejecución en una determinada máquina de referencia:

$$w_k \equiv \frac{C}{t_k^{REF}} \Rightarrow C = \frac{1}{\sum_{k=1}^n 1/t_k^{REF}}$$

Fórmula 4.2.2 Media geométrica

Dado un conjunto de n medidas, $S_1, \dots, S_n$, definimos su media geométrica:

$$\bar{S}_g = \sqrt[n]{\prod_{k=1}^n S_k}$$

Propiedad: cuando las medidas son ganancias en velocidad (*speedups*) con respecto a una máquina de referencia, este índice mantiene el mismo orden en las comparaciones independientemente de la máquina de referencia elegida (siempre que sea la misma en todos los casos).

$$SPEC(M) = \sqrt[n]{\frac{t_1^{REF}}{t_1^M} \times \dots \times \frac{t_n^{REF}}{t_n^M}} = \frac{\sqrt[n]{t_1^{REF} \times \dots \times t_n^{REF}}}{\sqrt[n]{t_1^M \times \dots \times t_n^M}} \Rightarrow$$

$$\frac{SPEC(MA)}{SPEC(MB)} = \frac{\sqrt[n]{t_1^{MB} \times \dots \times t_n^{MB}}}{\sqrt[n]{t_1^{MA} \times \dots \times t_n^{MA}}}$$

Luego, esencialmente, la máquina con mejor SPEC (SPEC mayor) siempre será la de menor media geométrica de los tiempos de ejecución de los programas del benchmark, independientemente de la máquina de referencia elegida (siempre que sea la misma en todos los casos).

$$SPEC(MA) > SPEC(MB) \Leftrightarrow \sqrt[n]{t_1^{MA} \times \dots \times t_n^{MA}} < \sqrt[n]{t_1^{MB} \times \dots \times t_n^{MB}}$$

Conclusión

- 1) La aritmética es fácilmente interpretable, el menor valor indica que ejecuta el **conjunto de programas** en un tiempo menor.
- 2) La aritmética ponderada nos permite **priorizar algunos programas**, debe usarse según lo que necesite cada usuario.
- 3) La media geométrica favorece mejoras sustanciales respecto a una máquina de referencia y no castiga al mismo nivel empeoramientos.

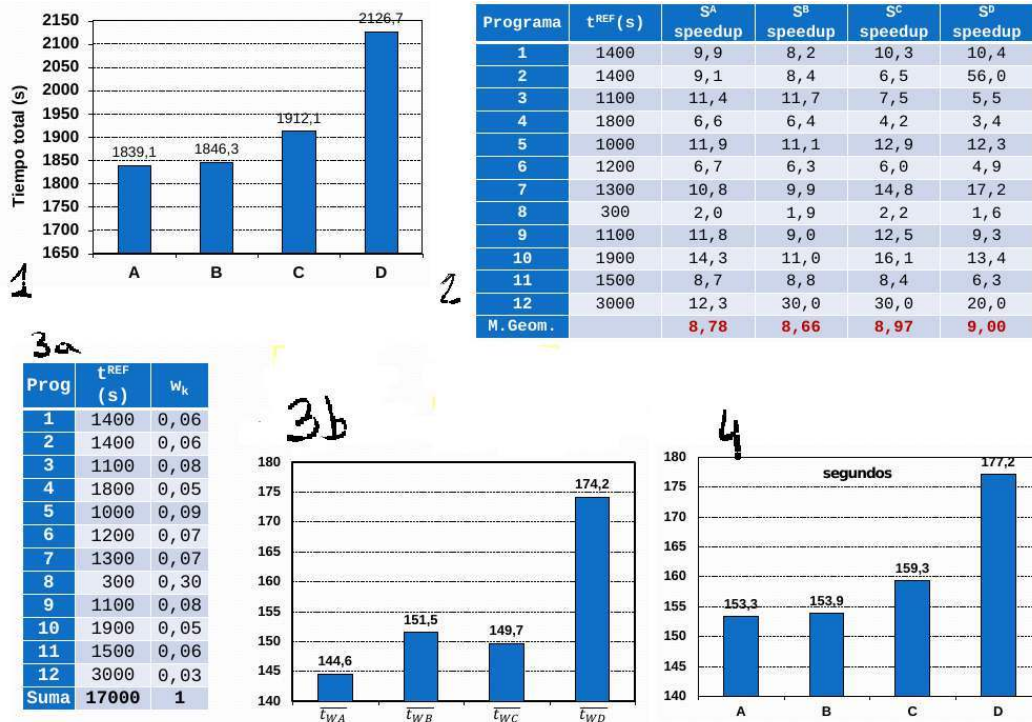


Figura 4.1: 1) Tiempo total de ejecución de 4 máquinas. 2) Índice speedup de las 4. 3a) Pesos de cada medida de referencia. 3b) Tiempos ponderados. 4) Media aritmética.

4.3

Comparación de prestaciones en presencia de aleatoriedad

Partimos de una tabla en la que tenemos una diferencia en tiempos de ejecución. Queremos saber si una es mejor que la otra, pero los tiempos de ejecución se ven afectados por **factores aleatorios**, así que necesitamos un método diferente a la media. Partimos de la **hipótesis nula** de que el rendimiento de las dos máquinas es equivalente, así que **las diferencias no deben ser significativas**.

4.3.1. Método 1

Tenemos un grado de significatividad α , que también te lo pueden dar en %. Si te dicen, por ejemplo, 95 %, entonces α es $1 - (95/100) = 0,05$. Es 0,05 si no te dicen nada. Para este método, nos dan un valor que se llama $p - value$. Si $p - value < \alpha$, rechazo hipótesis nula (son diferentes los servidores y uno es mejor que otro).

4.3.2. Método 2

Imaginemos que no sé el valor p . Me voy a la tabla. Busco el número $(t_{\frac{\alpha}{2}, n-1})$ de α y $\bar{d} = n - 1$ siendo n el número de valores que he medido. Si el tiempo experimental t_{exp} no pertenece a un intervalo de $[-t_{\frac{\alpha}{2}, n-1}, t_{\frac{\alpha}{2}, n-1}]$, entonces rechazo hipótesis nula.

4.3.3. Método 3

Imaginemos que no tenemos ni valor p ni tiempo experimental, pero tenemos un intervalo de confianza para $\bar{d}_{real}$. Este intervalo es:

$$\bar{d} - \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1}, \bar{d} + \frac{s}{\sqrt{n}} \times t_{\frac{\alpha}{2}, n-1}$$

Si 0 no pertenece a este intervalo, entonces rechazamos hipótesis nula.

Fórmula 4.3.1 Tiempo experimental

$$\text{Siendo } \bar{d} : t_{exp} = \frac{\bar{d} - \bar{d}_{real}}{s/\sqrt{n}} \sim T_{n-1}$$

$$\text{Siendo } s : \bar{d} = \frac{\sum_{i=1}^n d_i}{n}$$

$$\text{Siendo } s/\sqrt{n} \text{ el error estandar : } s = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n - 1}}$$

Nota ¿Me estudio las fórmulas?

Lo más importante es: **fórmula de SPEC, métodos de rechazar hipótesis nula y saber leer los valores que necesitamos en las imágenes que nos den**. En caso de que sí nos exigieran las fórmulas en el examen, no merece la pena, porque el temario es muy denso y este tema no vale tanto como el 2 o el 5. Invierte tu tiempo en estudiar las fórmulas del 5 y si te sobra tiempo después de todo, ven aquí y estúdiate esto.

4.3.4. Ejercicio con tabla y cómo interpretarla

Si nos dieran un ejercicio con una tabla ya hecha, tenemos que tener en cuenta los métodos de antes pero sin necesitar las fórmulas. En la imagen vemos que pvalue para la primera comparación es menor que α , pero que en la segunda no. Así que en la primera, son diferentes servidores y uno es mejor que el otro pero en la segunda no es significativo. Entonces, en la primera, vemos la diferencia. El primero menos el segundo da un valor positivo, así que el primer servidor tarda más tiempo que el segundo de media. Por lo tanto, el segundo servidor (Ej1_B) es mejor que el Ej1_A.

4.3.5. Estimar $\bar{d}_{real}$

Para estimar la media real y no depender de la muestral, podemos utilizar un intervalo de confianza que nos dice con una probabilidad de $(1 - \alpha)100\%$ donde podría estar. El intervalo es:

$$\bar{d} \pm \frac{s}{\sqrt{n}} t_{\frac{\alpha}{2}, n-1}$$

4.4

Diseño de experimentos de comparación de rendimiento

Definición 4.4.1 Variable respuesta o dependiente (=métrica)

El índice de rendimiento que usamos para las comparaciones. P.ej. tiempos de respuesta (R), productividades (X).

Definición 4.4.2 Factor

Cada una de las variables que pueden afectar a la variable respuesta. P.ej. sistema operativo, tamaño de memoria, tipo de disco duro, tipo de procesador, número de microprocesadores, número de cores, tamaño de cada caché, compilador, algún parámetro configurable del S.O. o del servidor, etc.

Definición 4.4.3 Nivel

Cada uno de los valores que puede asumir un factor. P.ej. para un S.O.: Windows, CentOS, Debian, Ubuntu; para un tipo de disco duro: SATA, IDE, SAS; para un parámetro del sistema operativo: ON, OFF, etc.

Definición 4.4.4 Interacción

Una interacción ocurre cuando el efecto de un factor cambia para diferentes niveles de otro factor. P.ej. el hecho de usar un tipo determinado de S.O. puede afectar a cómo de importante sea usar una mayor cantidad de memoria DRAM.

4.4.1. Tipos de diseños experimentales

- Diseños con **un solo factor**: Se utiliza una configuración determinada como base y se estudia un factor cada vez, midiendo los resultados para cada uno de sus niveles.
 - Problema: solo válida si descartamos que haya interacción entre factores.
- Diseños **multi-factoriales completos**: Se prueba cada posible combinación de niveles para todos los factores.
 - Ventaja: se analizan las interacciones entre todos los factores.
 - Número total de combinaciones $\prod_{i=1}^k n_i$, donde k es el número de factores y n_i el número de niveles del factor i -ésimo. En nuestro ejemplo: $4 \cdot 3 \cdot 3 = 36$ combinaciones.
- Diseños **multi-factoriales fraccionados**: No todas las interacciones se reflejan, solo las más probables.

4.4.2. Diseños con un solo factor

Para poder analizar si cambiar un componente hardware afecta a nuestro rendimiento, hacemos un experimentos con todas las posibles opciones. Para poder sacar si los cambios son significativos tendríamos que hacer un Test t para todos los cambios con todos (por ejemplo, si tenemos las interfaces de disco A, B y C tendríamos que hacer AB AC BC). Esto se vuelve cada vez más tedioso si queremos comparar muchas interfaces. Por eso, existe un test que nos **indica si el factor que estamos probando realmente importa**: el test **ANOVA** de un factor.

Tiene la **hipótesis nula** siguiente: El **factor no influye en el rendimiento**. Para saberlo, revisa si un valor pertenece a una determinada distribución (no hace falta saber cuál). Con su $p - value$, vemos si es menor que α . Si no lo es, el factor no influye a un $(1 - \alpha)100\%$ de confianza. Si influye, comparamos las medidas entre sí con test t. A esto se le llama **prueba de múltiples rangos/comparaciones múltiples**.

OPTIMIZACIÓN DEL RENDIMIENTO DE UN SERVIDOR MEDIANTE ANÁLISIS OPERACIONAL

5

5.1

Introducción: Redes de Colas de Espera

Definición 5.1.1 Redes de colas de espera

Abstracción de un sistema informático real, en el que tenemos un **conjunto de dispositivos** (= recursos físicos) interrelacionados y **trabajos** que los usan (carga)

Normalmente un dispositivo o recurso solo puede ser usado por un trabajo a la vez. El resto de trabajos tendrá que esperar.

Modelos basados en redes de colas (queueing networks):

- Una red de colas está formada por un conjunto de estaciones de servicio conectadas entre sí.
- **Estación de servicio** (*service station*): Objeto compuesto por un dispositivo (recurso físico) que presta un servicio y una cola de espera para los trabajos (clientes) que demandan un servicio de él.

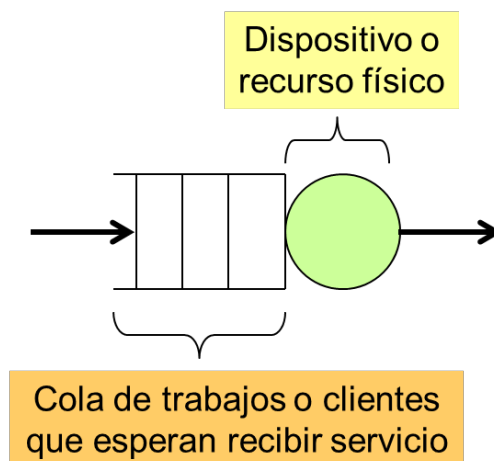


Figura 5.1: Concepto de estación de servicio

5.1.1. El modelo de servidor central

Es la red de colas que más se ha utilizado para representar el comportamiento básico de los programas en un servidor de cara a extraer información sobre su rendimiento.

Un trabajo llega al servidor, comienza usando la CPU. A continuación puede terminar (sale del servidor) o realizar un acceso a una unidad de E/S. Después de una operación con una unidad de E/S, el trabajo vuelve a la CPU.

Variables que caracterizan a un trabajo en una estación de servicio

- **Tiempo de espera en cola** (W , *waiting time*): Tiempo transcurrido desde que el trabajo solicita hacer uso del recurso físico (=se pone en la cola) hasta que realmente empieza a utilizarlo.
- **Tiempo de servicio** (S , *service time*): Desde que el trabajo accede al recurso físico hasta que lo libera (=tiempo que tarda el recurso físico en procesar el trabajo).
- **Tiempo de respuesta** (R , *response time*): Suma de los dos tiempos anteriores $R = W + S$.

En las estaciones con más de un servidor, cuantos más recursos el tiempo de espera en cola (W) es menor. Con infinitos dispositivos $W = 0$, $R = S$.

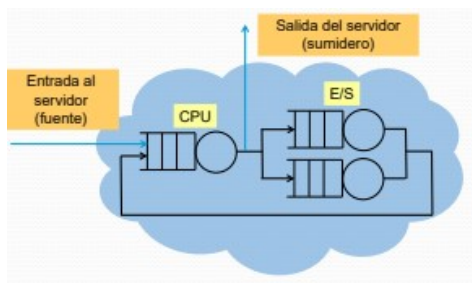


Figura 5.2: Modelo de servidor central.

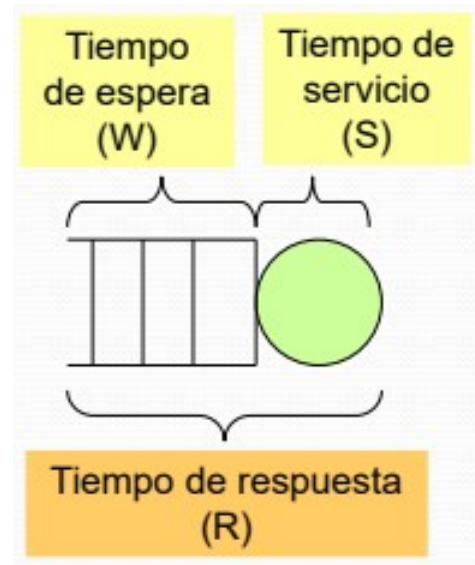


Figura 5.3: Variables que caracterizan a un trabajo en una estación de servicio.

Definición 5.1.2 Tiempo de reflexión (Z)

Es un parámetro que representa el tiempo que requiere el cliente antes de volver a lanzar una petición al servidor tras la respuesta de este. Se suele modelar mediante una estación de servicio tipo retardo con un tiempo de servicio = Z . Para ello, realizamos una hipótesis adicional: **cada cliente envía un único trabajo al servidor.**

5.1.2. Redes de colas cerradas

Presentan un número constante de trabajos que van **recirculando** por la red (N_T). Dependiendo de si hay o no interacción con los clientes se distingue entre redes de tipo **batch** (por lotes) o redes **interactivas**.

Siempre supondremos 1 cliente = 1 trabajo:

- N_0 = Número de trabajos en el servidor = Número de clientes conectados al servidor
- N_Z = Números de clientes en reflexión = Número de trabajos en “reflección” (= esperando a que los clientes vuelvan a introducirlos en el servidor)

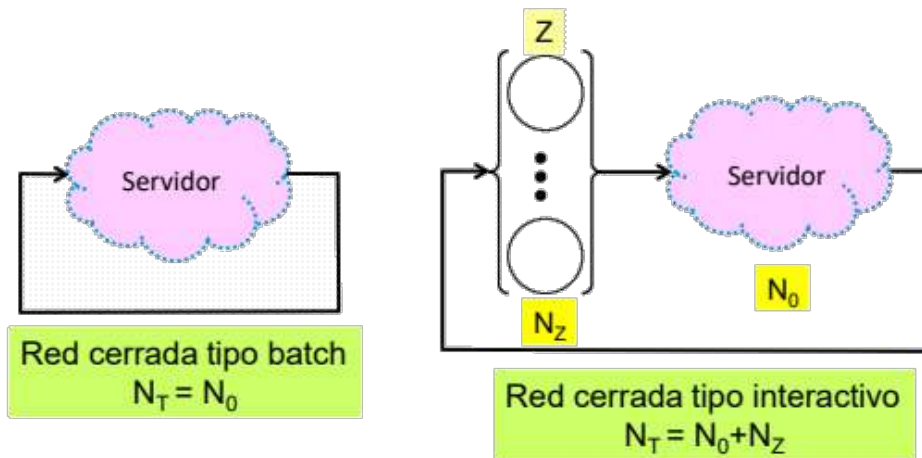


Figura 5.4: A la derecha: redes por lotes (*batch*), a la izquierda: redes interactivas

5.1.3. Redes de colas abiertas

Los trabajos llegan a la red a través de una fuente externa que no controlamos. Tras ser procesados, salen de ella a través de uno o más sumideros. **No existe realimentación entre sumidero y fuente.**

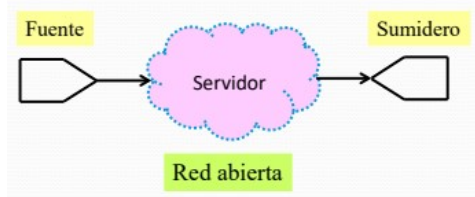


Figura 5.5: Redes de cola s abiertas.

5.1.4. Redes mixtas

Cuando el modelo no corresponde a ninguno de los dos anteriores.

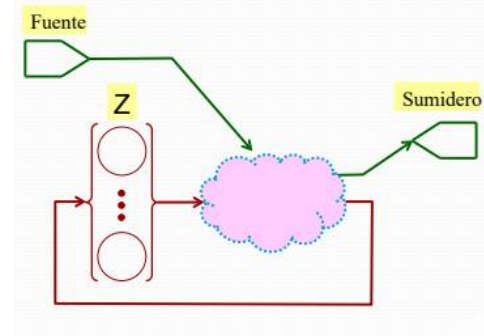


Figura 5.6: Redes de colas mixtas.

5.2.1. Análisis operacional

Definición 5.2.1 Análisis operacional

Técnica de análisis de redes de colas basada en **valores medios** de diferentes variables medibles (= variables operacionales) del servidor.

- Nos proporcionará relaciones generales entre las variables operacionales (leyes operacionales).
- Nos permitirá identificar el cuello de botella del servidor y estimar los límites de sus prestaciones cuando aumente la carga (= capacidad del servidor).
- Nos permitirá evaluar los efectos en el rendimiento de diferentes modificaciones en los recursos del servidor.

5.2.2. Variables del servidor

- El servidor contiene K estaciones de servicio (una por cada dispositivo o recurso físico del servidor que influya en sus prestaciones).
- A todo el servidor en su globalidad lo denotamos como dispositivo “cero”.

5.2.3. Variables operacionales básicas

- **Variable global temporal** (T): Duración del periodo de medida para el que se extrae el modelo.
- Variables operacionales básicas de la estación de servicio i -ésima **medidas durante el periodo de medida**:
 - A_i : Número de **trabajos solicitados** a la estación i -ésima (llegadas, *arrivals*).
 - B_i : Tiempo que el recurso físico de la estación i -ésima **ha estado en uso** (= ocupado, *busy time*).
 - C_i : Número de **trabajos completados** por la estación i -ésima (salidas, *completions*).

5.2.4. Variables operacionales deducidas

Todas ellas son valores medios y muchas se obtienen fácilmente a partir de las variables operacionales básicas:

- S_i Tiempo medio de servicio (*service time*).
- W_i Tiempo medio de espera en cola (*waiting time*).
- R_i Tiempo medio de respuesta (*response time*).
- λ_i Tasa media de llegada (*arrival rate*).
- X_i Productividad media (*throughput*).
- U_i Utilización (*utilization*): fracción del tiempo T que el recurso físico de la estación i -ésima ha estado en uso (*busy*).

$$U_i = \frac{B_i}{T}$$

También hay variables operaciones deducidas que hacen referencia al número medio de trabajos, durante el periodo de medida T , en cada parte de una estación de servicio:

- N_i Número medio de trabajos en la estación de servicio i -ésima (cola más recurso físico).
- Q_i Número medio de trabajos en cola de espera de la estación i -ésima (*jobs in queue*).
- U_i Número medio de trabajos siendo servidos por el recurso físico de la estación i -ésima, $U_i = N_i - Q_i$. Coincide numéricamente con su utilización: $U_i = \frac{B_i}{T}$

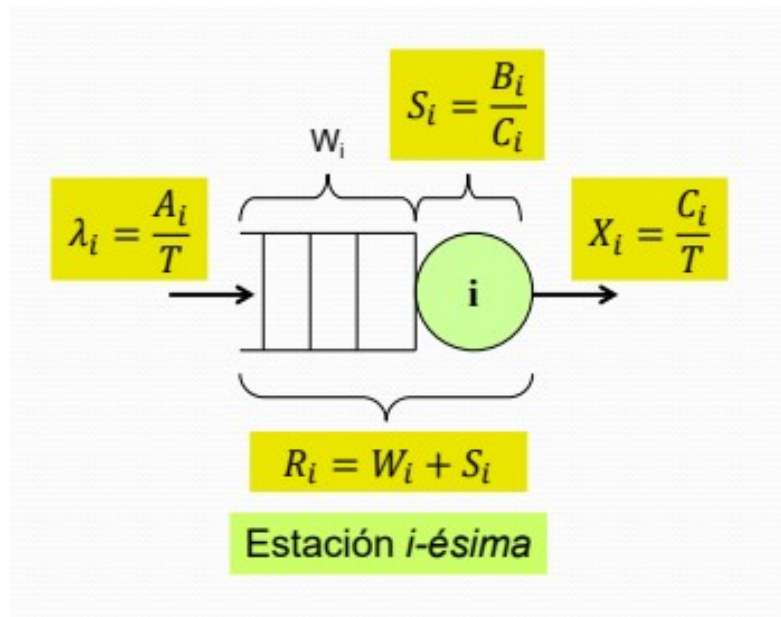


Figura 5.7: Variables operacionales

5.2.5. Variables operacionales de un servidor

Variables operacionales básicas de un servidor (todas referidas al periodo de medida T):

- A_0 Número de trabajos solicitados al servidor (*arrivals*).
- C_0 Número de trabajos completados por el servidor (*completions*).

Variables operacionales deducidas de un servidor (todas referidas al periodo de medida T):

- λ_0 Tasa media de llegada al servidor (*arrival rate*).
- X_0 Productividad media del servidor (*throughput*).
- N_0 Número medio de trabajos en el servidor (= número medio de clientes conectados al servidor, ya que suponemos que cada cliente envía un único trabajo al servidor). $N_0 = N_1 + N_2 + \dots + N_K$
- R_0 Tiempo medio de respuesta del servidor (*response time*) $\equiv$ tiempo que tarda, de media, el servidor en procesar una petición.

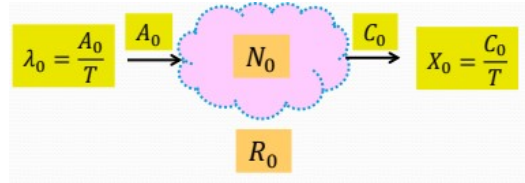


Figura 5.8: Variables operacionales de un servidor

5.2.6. Razón de visita y demanda de servicio

Fórmula 5.2.1 Razón media de visita V_i (*visit ratio*)

Representa la proporción entre el número de trabajos completados por el servidor y el número de trabajos completados por la estación de servicio i -ésima. Nos indica el número de veces que, de media, un trabajo "visita" la estación de servicio i -ésima antes de abandonar el servidor.

$$V_i = \frac{C_i}{C_0}$$

Fórmula 5.2.2 Demanda media de servicio D_i (*service demand*)

Cantidad de tiempo que, por término i medio, el recurso físico de la estación de servicio i -ésima le ha dedicado a cada trabajo que abandona el servidor (= que ha sido procesado por completo por el servidor).

$$D_i = \frac{B_i}{C_0} = V_i \times S_i$$

5.2.7. Leyes operacionales y equilibrio de flujo

Existen una serie de relaciones entre algunas variables operacionales que se mantienen válidas para cualquier intervalo de observación y que no dependen de suposiciones sobre la distribución de los tiempos de servicio o de la forma en la que llegan los trabajos. Estas relaciones se denominan **leyes operacionales**.

Estas leyes se pueden expresar de una forma alternativa cuando el servidor se encuentra en **equilibrio de flujo**. Si se escoge un intervalo de observación T suficientemente largo (con respecto a lo que suele tardar el servidor en servir una petición, R_0), diremos que un servidor está en **equilibrio de flujo** si se cumple que:

- El número de trabajos que completa el servidor coincide aproximadamente con los solicitados ($C_0 \approx A_0$). Dicho de otra manera, la productividad media coincide aproximadamente con la tasa media de llegada ($X_0 \approx \lambda_0$).
- El número de trabajos que completa cada estación de servicio coincide aproximadamente con los que se solicitan: ($C_i \approx A_i \Rightarrow X_i \approx \lambda_i, \forall i = 1 \dots K$)

Fórmula 5.2.3 Ley de la Utilización

Relaciona la utilización de un recurso físico con el número de trabajos que completa por unidad de tiempo (= su productividad media) y el tiempo que le dedica, de media, a cada uno de ellos (= su tiempo de servicio medio):

$$\forall i = 1 \dots K \quad U_i = X_i \times S_i = \lambda_i \times S_i$$
$$S_i = \frac{B_i}{C_i} = \frac{B_i/T}{C_i/T} = \frac{U_i}{X_i}$$

Una consecuencia inmediata de esta ley es que la productividad media de un dispositivo viene limitada por la inversa de su tiempo de servicio:

$$U_i \leq 1 \Rightarrow X_i \leq \frac{1}{S_i} \forall i = 1 \dots K$$

Fórmula 5.2.4 Ley del flujo forzado y relación utilización-demanda

La **ley del flujo forzado** relaciona la productividad del servidor con la de cada una de las estaciones que integran el mismo:

$$\forall i = 1 \dots K \quad X_i = X_0 \times V_i = \lambda_0 \times V_i = \lambda_i$$
$$V_i = \frac{C_i}{C_0} = \frac{C_i/T}{C_0/T} = \frac{X_i}{X_0}$$

La utilización de cada dispositivo o recurso físico es proporcional a su demanda de servicio, siendo la constante de proporcionalidad precisamente la productividad global del servidor (**relación utilización-demanda de servicio**):

$$\forall i = 1 \dots K \quad U_i = X_0 \times D_i = \lambda_0 \times D_i$$
$$D_i = \frac{B_i}{C_0} = \frac{B_i/T}{C_0/T} = \frac{U_i}{X_0}$$
$$U_i \leq 1 \Rightarrow X_0 \leq \frac{1}{D_i} \forall i = 1 \dots K$$

Fórmula 5.2.5 Ley de Little

Aplicada a un servidor, esta ley relaciona las dos variables más importantes que reflejan el rendimiento de un servidor: su productividad media (X_0) y su tiempo medio de respuesta (R_0). Esta ley solo es válida cuando el servidor está en equilibrio de flujo.

$$N_0 = \lambda_0 \times R_0 = X_0 \times R_0$$

La ley de Little puede ser aplicada no solo al servidor en su totalidad, sino a cada estación de servicio y a cada uno de los diferentes sub-niveles de una estación de servicio.

- Aplicación a toda una estación de servicio:

$$N_i = \lambda_i \times R_i = X_i \times R_i$$

- Aplicación a la cola de una estación de servicio:

$$Q_i = \lambda_i \times W_i = X_i \times W_i$$

Fórmula 5.2.6 Ley general del tiempo de respuesta

El tiempo medio de respuesta que experimenta, de media, una petición a un servidor en equilibrio de flujo se puede calcular teniendo en cuenta que cada una de ellas ha tenido que "visitar" V_i veces a la estación i -ésima, requiriendo cada visita una media de R_i segundos:

$$R_0 = V_1 \times R_1 + V_2 \times R_2 + \dots + V_K \times R_K = \sum_{i=1}^K V_i \times R_i$$

$$\begin{aligned} N_0 &= N_1 + N_2 + \dots + N_K \xrightarrow{\text{Ley de Little}} X_0 \times R_0 = X_1 \times R_1 + X_2 \times R_2 + \dots + X_K \times R_K \\ \text{Ley del Flujo Forzado} \Rightarrow X_0 \times R_0 &= X_0 \times V_1 \times R_1 + X_0 \times V_2 \times R_2 + \dots + X_0 \times V_K \times R_K \end{aligned}$$

Figura 5.9: Demostración de la Ley general del tiempo de respuesta

Fórmula 5.2.7 Ley del tiempo de respuesta interactivo

En una red de colas cerrada el número total de trabajos (= clientes) en la red ($N_T = N_Z + N_0$) es constante. Siempre supondremos que el tamaño de las colas es suficientemente grande ($\geq N_T$) para asegurar que no se pierde ningún trabajo, por lo que **un servidor en una red de colas cerrada siempre está en equilibrio de flujo**.

Podemos, entonces, aplicar la ley de Little a diversas partes de la red de colas:

- Ley de Little aplicada a los clientes en reflexión: $N_Z = X_0 \times Z$, donde N_Z = Número medio de clientes (= trabajos) en reflexión.
- Ley de Little aplicada al servidor: $N_0 = X_0 \times R_0$

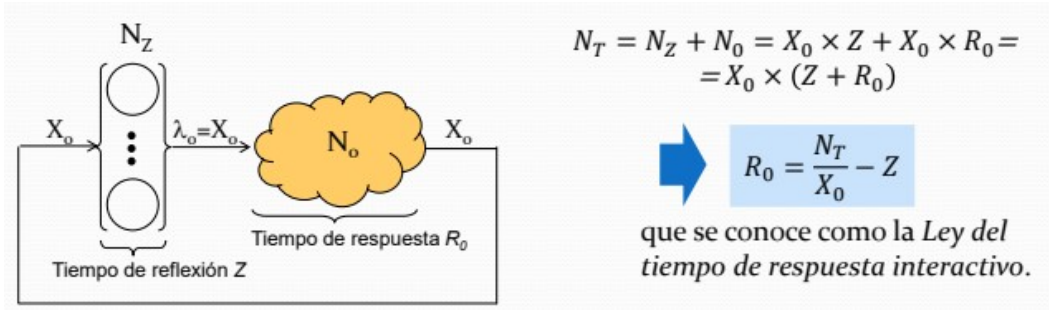


Figura 5.10: Ley del tiempo de respuesta interactivo

5.3 Límites optimistas del rendimiento

5.3.1. Limitaciones en el rendimiento: cuello de botella

Todo servidor presenta alguna limitación en su rendimiento. La localización del elemento limitador no solo depende del servidor sino también del tipo carga.

- Al elemento limitador del rendimiento del servidor se le denomina cuello de botella (*bottleneck*).
- Además, puede haber más de uno de estos elementos limitadores.

Veremos que la única manera de mejorar las prestaciones de un servidor de manera significativa es actuando sobre el cuello de botella.

Identificación del cuello de botella

El cuello de botella es el dispositivo o recurso físico que primero llegará a saturarse (utilización = 1) cuando aumente la carga (λ_0 mayor).

$$U_i = X_0 \times D_i = \lambda_0 \times D_i \quad \forall i = 1 \dots K$$

Como U_i es proporcional a D_i , podemos identificar fácilmente el cuello de botella de un servidor simplemente identificando el dispositivo con mayor demanda de servicio o con mayor utilización.

No hace falta llevar el servidor al límite para identificar el cuello de botella.

Como $D_i = V_i \times S_i$, concluimos que la localización del cuello de botella no solo depende de lo rápido que sea el recurso físico (S_i) sino también del tipo de carga a la que está sometido (V_i).

Denotaremos por "b" (*bottleneck*) al índice de la estación de servicio que contiene el cuello de botella. Su demanda de servicio y su utilización vendrán dadas por.

$$D_b = \max_{i=1 \dots K} \{D_i\} = V_b \times S_b \quad U_b = \max_{i=1 \dots K} \{U_i\} = X_0 \times D_b$$

5.3.2. Saturación del servidor

El servidor se satura cuando lo hace el cuello de botella ya que éste será el primer recurso físico en alcanzar una utilización = 1 cuando aumente la carga. El equilibrio de flujo siempre se cumple mientras el servidor no esté saturado. En saturación, el cuello de botella está al máximo de su productividad:

$$1 = U_b = X_b \times S_b \Rightarrow X_b = 1/S_b$$

Definición 5.3.1 Servidor equilibrado (*balanced system*)

Servidor en el que todos los recursos físicos, de media, tienen la misma demanda de servicio y utilización (la carga se absorbe equitativamente):

$$U_i \approx U_j \quad \forall i, j = 1 \dots K \Rightarrow D_i \approx D_j \quad \forall i, j = 1 \dots K$$

5.3.3. Límites optimistas: redes abiertas

El valor máximo de la productividad media del servidor será aquél producido por una tasa de llegada que sature el cuello de botella ($U_b = 1$):

$$U_b = X_0 \times D_b = \lambda_0 \times D_b \quad Si \quad U_b = 1 \Rightarrow X_0^{max} = \frac{1}{D_b}$$

Una tasa de llegada mayor provocaría un aumento descontrolado de la cola del cuello de botella hasta su desbordamiento final y, por tanto, dejaría de cumplirse la hipótesis del equilibrio de flujo. R_0 crecería igualmente sin control. Si $\lambda_0 < 1/D_b$ **el servidor no estará saturado y, por tanto, estará en equilibrio de flujo**.

El valor más optimista (= el valor mínimo) del tiempo medio de respuesta del servidor (R_0^{min}) será el que experimenta un trabajo cuando llega al servidor sin que haya otros trabajos previamente ($W_i = 0 \quad \forall i = 1, \dots, K$):

$$R_0 = \sum_{i=1}^K V_i \times R_i = \sum_{i=1}^K V_i \times (W_i + S_i) \Rightarrow R_0 \rightarrow R_0^{min} = \sum_{i=1}^K V_i \times S_i = \sum_{i=1}^K D_i \equiv D$$

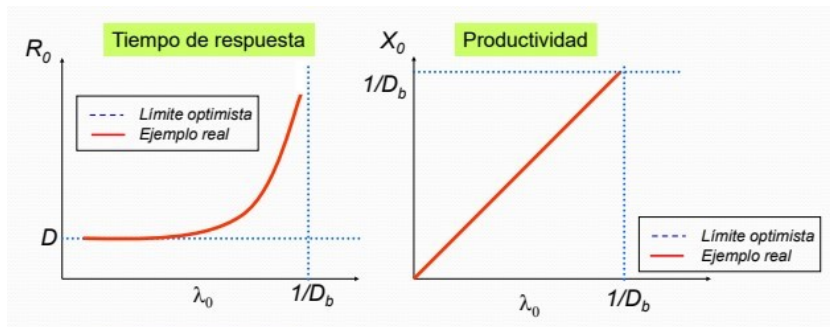


Figura 5.11: Límites optimistas en redes abiertas

5.3.4. Límites optimistas: redes cerradas

Fórmula 5.3.1 Ley de Little a la red completa ($N_T = N_0 + N_Z$)

$$N_T = X_0 \times (R_0 + Z) \Rightarrow R_0 = \frac{N_T}{X_0} - Z$$

Para valores de carga altos (N_T grande)

Valor optimista de la productividad media: Cuando el cuello de botella esté cerca de la saturación:

$$U_b = X_0 \times D_b \quad Si \ U_b \rightarrow 1 \Rightarrow X_0 \rightarrow X_0^{max} = \frac{1}{D_b}$$

Valor optimista del tiempo de respuesta medio: A partir del valor optimista de la productividad media (sin más que reemplazar ese valor de X_0 en la ley de Little a la red completa):

$$R_0 \rightarrow \left(\frac{N_T}{X_0^{max}} \right) - Z = D_b \times N_T - Z$$

Para valores de carga bajos (N_T pequeño)

Valor optimista del tiempo de respuesta medio: cuando los trabajos siempre encuentran los dispositivos sin ocupar ($W_i = 0 \ \forall i = 1..K$):

$$R_0 = \sum_{i=1}^K V_i \times R_i = \sum_{i=1}^K V_i \times (W_i + S_i) \Rightarrow R_0 \rightarrow R_0^{min} = \sum_{i=1}^K V_i \times S_i = \sum_{i=1}^K D_i \equiv D$$

Valor optimista de la productividad media: A partir del valor optimista del tiempo de respuesta medio (sin más que reemplazar ese valor de R_0 en la ley de Little a la red completa):

$$X_0 \rightarrow \frac{N_T}{R_0^{min} + Z} = \frac{N_T}{D + Z}$$

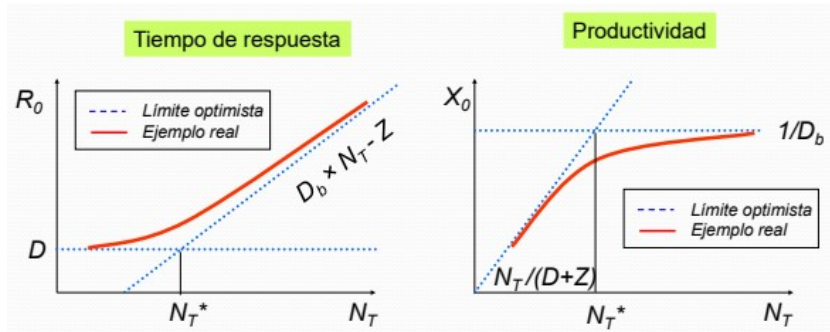


Figura 5.12: Límites optimistas en redes cerradas

5.3.5. Punto de saturación (*knee point*)

Es el valor de N_T en donde las asíntotas coinciden:

$$D = D_b \times N_T^* - Z \Rightarrow N_T^* = \frac{D + Z}{D_b}$$

Propiedades
del punto teó-
rico de sa-
turación N_T^*

- Para un número total de trabajos $N_T > N_T^*$, los límites en las prestaciones vienen impuestos únicamente por el cuello de botella del servidor.
- A partir de N_T^* trabajos ya no se puede conseguir el tiempo de respuesta mínimo ya que se empiezan a formar colas de espera en, al menos, el cuello de botella (en la práctica, esto sucede bastante antes).
- En principio, podría parecer el número ideal de trabajos en la red ya que, al menos teóricamente, para $N_T = N_T^*$ se podría conseguir la productividad máxima y el tiempo de respuesta mínimo absolutos del servidor (en la práctica esto nunca se puede conseguir de forma simultánea): $N_T^* = X_0^{max} \times (R_0^{min} + Z) = \frac{D+Z}{D_b}$

5.4 Técnicas de mejora

Para mejorar las prestaciones de manera significativa hay que actuar sobre el cuello de botella del servidor: **hay que reducir** $D_b = V_b \times S_b$

5.4.1. Sintonización o ajuste (*Tuning*)

Optimizas los componentes que ya tienes con parámetros, ficheros de configuración y políticas. Puede alterar la fiabilidad y hay que hacer tests y conocer muy bien el sistema con el que trabajamos.

5.4.2. Actualización y/o ampliación

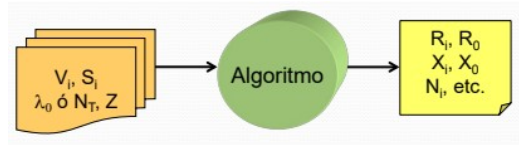
Reemplazamos los dispositivos por otros más rápidos (menos Si). Añadimos más dispositivos (Menos Vi). Depende de la facilidad del servidor para **escalar** y debe ser **compatible con lo que ya tenemos**.

5.5

Algoritmos de resolución de modelos de redes de colas

En este apartado vamos a proporcionar una metodología (algoritmo) para resolver modelos de redes de colas. Supondremos conocido:

- El número de estaciones de servicio (K).
- Por cada estación:
 - Razón de visita medio de cada estación ($V_i = C_i/C_0$).
 - Tiempo de servicio medio de cada estación ($S_i = B_i/C_i$).
- Si la red es abierta: Tasa de llegada al servidor ($\lambda_0 = A_0/T$).
- Si la red es cerrada:
 - Número total de trabajos en el sistema servidor+clientes (N_T).
 - Tiempo medio de reflexión de los clientes (Z).



5.5.1. Resolución de Redes abiertas

Si los trabajos son independientes entre sí, está en equilibrio de flujo y es una red abierta:

$$W_i = N_i \times S_i$$

Entonces, el **tiempo de respuesta**:

$$R_i = W_i + S_i = N_i \times S_i + S_i$$

Si aplicamos la **Ley de Little** ($N_i = X_i \times R_i$):

$$R_i = N_i \times S_i + S_i = X_i \times R_i \times S_i + S_i \Rightarrow \frac{S_i}{1 - X_i \times S_i} = \frac{S_i}{1 - U_i} = \frac{S_i}{1 - X_0 \times D_i} = \frac{S_i}{1 - \lambda_0 \times V_i \times S_i}$$

A continuación, calculamos el tiempo medio de respuesta del servidor (R_0):

$$R_0 = \sum_{i=1}^K V_i \times R_i = \sum_{i=1}^K \frac{S_i \times V_i}{1 - \lambda_0 \times V_i \times S_i} = \sum_{i=1}^K \frac{D_i}{1 - \lambda_0 \times D_i}$$

El resto de variables operacionales se pueden calcular fácilmente usando sus expresiones habituales.

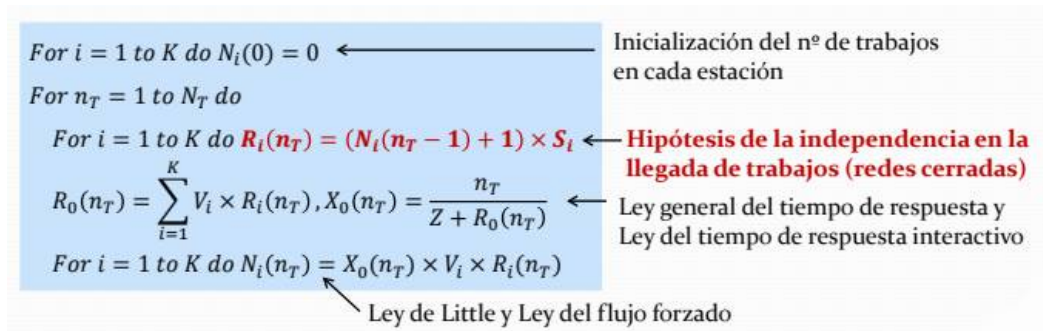
5.5.2. Resolución de redes cerradas

Suponemos conocidos V_i , S_i , N_T y Z . Estamos en **equilibrio de flujo**.

Método: debemos ir resolviendo la red para valores incrementales del número de trabajos en la red hasta alcanzar $N_T : n_T = 1, \dots, N_T$.

Notación: $N_i(n_T)$: Número de trabajos en la estación i -ésima si en la red hubiese n_T trabajos. Ídem para los tiempos de respuesta $R_i(n_T)$ y las productividades $X_i(n_T)$.

Hipótesis de la independencia en la llegada de trabajos para redes cerradas: $W_i(n_T) = N_i(n_T - 1) \times S_i \Rightarrow R_i(n_T) = W_i(n_T) + S_i = (N_i(n_T - 1) + 1) \times S_i$



PLIEGOS DE PRESCRIPCIONES TÉCNICAS

6

6.1 Introducción

Definición 6.1.1 Licitación

Subasta para la adjudicación de un contrato, generalmente de servicio público, como la ejecución de una obra, el suministro de provisiones, etc., a quien presente la propuesta más ventajosa.

Definición 6.1.2 Contrato

Documento que recoge las condiciones de un pacto o convenio entre partes que se obligan para la ejecución de un determinado trabajo, y a cuyo cumplimiento pueden ser compelidas.

Definición 6.1.3 Licitador

El que participa en una licitación ofreciendo la ejecución de un servicio a cambio de la obtención de dinero u otros beneficios.

Definición 6.1.4 Contratista

Es el licitador que resulta adjudicatario de la licitación y quien finalmente se encargará de llevar a cabo los trabajos contratados.

Definición 6.1.5 Contratante

El que contrata. Normalmente, el que ha propuesto la licitación.

Tipos de contratos

- **Contrato de obras.** Aquéllos que tienen por objeto la ejecución de una obra. Por «obra» se entenderá el resultado de un conjunto de trabajos de construcción o de ingeniería civil, destinado a cumplir por sí mismo una función económica o técnica, que tenga por objeto un bien inmueble.
- **Contrato de suministro.** Los que tienen por objeto la adquisición o el arrendamiento, con o sin opción de compra, de productos o bienes muebles.
- **Contrato de servicios.** Aquéllos cuyo objeto son prestaciones de hacer consistentes en el desarrollo de una actividad o dirigidas a la obtención de un resultado distinto de una obra o un suministro. Ej. servicios de mantenimiento y reparación, servicios de uso de recursos informáticos.

Se abre la licitación y se publica el expediente con el pliego de condiciones. Se anuncian en el BOE (Boletín Oficial del Estado). En los anuncios debe figurar:

- Identificación del contratante
- Descripción del tipo de contrato.
- Presupuesto total o estimado. Si está dividido en lotes, por cada lote.
- Calendario para la entrega y duración del contrato.
- Criterios para adjudicar el contrato.
- Plazo para las ofertas.
- Códigos CPV (vocabulario común de contratos públicos). Clasifican actividades económicas para contratos de licitación en la UE.
- Dirección en Internet del pliego de condiciones.

6.2

El pliego de condiciones

Regulan derechos, responsabilidades, obligaciones y garantías entre los agentes del contrato. Tiene las exigencias jurídicas, económicas, administrativas y técnicas para el trabajo. Se divide en: PCAP y PPT

6.2.1. Pliego de Cláusulas Administrativas Particulares (PCAP)

Aquellas declaraciones **jurídicas, económicas y administrativas** que sean específicas del contrato de que se trate o del procedimiento de adjudicación del mismo.

Los pliegos de cláusulas administrativas particulares deberán contener con carácter general para todos los contratos los siguientes datos:

- Objeto del contrato y sus códigos CPV.
- Presupuesto de la licitación.
- Plazo de ejecución o de duración del contrato.
- Procedimiento y forma de adjudicación del contrato.
- Documentos que presentar por los licitadores.
- Derechos y obligaciones de las partes.
- Régimen de pagos.

Otra información que suele contener: confidencialidad, incidencias en la ejecución del contrato (p.ej. motivos de resolución del contrato, garantías exigidas al contratista), inspección de la contratación, etc.

6.2.2. Pliego de Prescripciones Técnicas (PPT)

El pliego de prescripciones técnicas contendrá, al menos, los siguientes extremos:

- Descripción y especificaciones técnicas de los bienes o prestaciones del contrato (Características exigidas del servicio).
- En su caso, descripción y especificaciones técnicas de las variantes.
- Precio de cada una de las unidades (lotes) en que se descompone el presupuesto y número de unidades a suministrar.

En ningún caso contendrán estos pliegos declaraciones o cláusulas que deban figurar en el pliego de cláusulas administrativas particulares.

Reglas para redactar un pliego de prescripciones técnicas

Con carácter general, se definirán aplicando criterios de **sostenibilidad, protección ambiental, accesibilidad universal e igualdad de oportunidades**.

No harán referencia a una fabricación o una procedencia determinada con la finalidad de favorecer o descartar ciertas empresas o ciertos productos. Si no es posible, se acompañará la mención «o equivalente».

Se formularán de alguna de las siguientes maneras o una mezcla de ellas:

- En términos de rendimiento o de requisitos funcionales.
- Por referencia a especificaciones técnicas de normas nacionales o internacionales.
- **Objeto.** Especifica el contexto en el que se ubica el pliego de prescripciones y su objetivo principal.
- **Descripción general** de la obra/suministro/servicio a realizar, incluidas las posibles variantes.
- **Especificaciones técnicas mínimas exigidas** a cada uno de los elementos de la obra/suministro/servicio a realizar y de sus variantes.
- **Precio** de cada una de las unidades (lotes) en que se descompone el presupuesto y número de unidades a suministrar.
- **Garantías** de los componentes y de las obras realizadas.
- **Servicio de ayuda/soporte técnico y/o mantenimiento.** Tipo de servicio, disponibilidad, precio, etc.
- **Documentos, manuales y formación.** Sobre la documentación que se debe aportar para el uso y mantenimiento de los bienes instalados. Incluso sobre la formación del personal de la empresa.
- **Instalación y puesta a punto.** Los requisitos que se deben cumplir para considerar que lo contratado se ha instalado correctamente.

Algunas secciones habituales en un PPT

Las proposiciones de los interesados deberán ajustarse a lo previsto en el pliego de condiciones, y su presentación supone la aceptación incondicionada por el empresario del contenido de la totalidad de dichas cláusulas o condiciones, sin salvedad o reserva alguna.

Partes habituales de una oferta

- Documentación sobre la empresa donde se verifique que cumple los requisitos legales y administrativos de los pliegos.
- Memoria técnica con la solución propuesta.
- Precio por el que se compromete a realizar los trabajos indicados en la memoria técnica.

Cuando sólo se utilice un criterio de adjudicación, éste ha de ser el del precio más bajo. Los criterios que han de servir de base para la adjudicación del contrato se determinarán por el contratante y se detallarán en los pliegos de cláusulas administrativas particulares.

No podrá declararse desierta una licitación cuando exista alguna oferta o proposición que sea admisible de acuerdo con los criterios que figuren en el pliego.

El documento de formalización será firmado tanto por la entidad adjudicadora (contratante) como por el adjudicatario (contratista) y **se unirá al mismo**, como anexo, **un ejemplar del pliego de condiciones**.

El documento contractual no podrá incluir estipulaciones que establezcan derechos y obligaciones para las partes, distintos de los previstos en los pliegos.

En el contrato deben figurar, salvo que ya se encuentren recogidas en los pliegos, las siguientes menciones:

- La identificación de las partes.
- Definición del objeto y tipo del contrato.
- Referencia a la legislación aplicable al contrato.
- La cuantía del contrato, su duración o las fechas estimadas para el comienzo de su ejecución y para su finalización, así como la de la prórroga o prórrogas, si estuviesen previstas.
- Las condiciones de pago.
- Los supuestos en que procede la modificación y/o la resolución del contrato.

Fórmulas Ingeniería de servidores

@mariouwu

Tema 1: Introducción a la ingeniería de servidores

Ganancia en velocidad

$$S_b(A) = \frac{V_b}{V_a} = \frac{t_a}{t_b} \longrightarrow t_a \text{ es el tiempo que tarda en ejecutarse el programa en la máquina a}$$

$\longrightarrow V_b$ es la velocidad de la máquina b

$$\text{Prestaciones / Coste} = \frac{V_a}{\text{Coste}_a} = \frac{\frac{1}{T_a}}{\text{Coste}_a} = \frac{1}{T_a \cdot \text{Coste}_a} \implies \text{Utilizando estos datos, podemos obtener la ganancia en prestaciones de A con respecto a B :}$$

$$\text{Mejora en velocidad} = \frac{V_b - V_a}{V_a} \cdot 100 = (S_b(A) - 1) \cdot 100$$

$$S_b(A) = \frac{T_a \cdot \text{Coste}_a}{T_b \cdot \text{Coste}_b}$$

Ley de Amdahl

$$\text{Mejora del tiempo de ejecución de un proceso} \left\{ \begin{array}{l} T_{\text{original}} = f_o \cdot t_o + (1 - f) \cdot t_o \\ T_{\text{mejora}} = \frac{f_o \cdot t_o}{k} + (1 - f) \cdot t_o \end{array} \right.$$

$$S \equiv S_o(m) = \frac{V_m}{V_o} = \frac{T_o}{T_m} = \frac{T_o}{(1-f) \cdot T_o + \frac{f \cdot T_o}{k}}$$

$$\text{Casos particulares de la Ley de Amdahl} \longrightarrow \begin{array}{l} \text{Si } f=0 \longrightarrow S=1 \text{ (No hay mejora)} \\ \text{Si } f=1 \longrightarrow S=k \text{ (Mejora k veces)} \\ \text{Si } k=\infty \longrightarrow S \approx \frac{1}{1-f} \end{array}$$

$$\text{Si sacamos factor común } T_o \text{ y despejamos } S \equiv S_o(m) = \frac{1}{1-f + \frac{f}{k}} \downarrow \text{Ganancia respecto a la original}$$

$f \equiv$ fracción de tiempo usado por el recurso

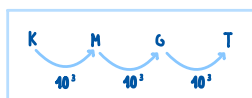
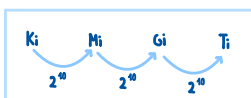
Tema 3: monitorización de servicios y programas

Sobrecarga (Overhead)

$$\text{Sobrecarga}_{\text{recurso}} (\%) = \frac{\text{Uso del recurso por parte del monitor}}{\text{Capacidad total del recurso}} \cdot 100 \implies \frac{T_{\text{EZ}}}{T_P} \cdot 100$$

cuanto se activa
Cada cuanto se activa

Factores de conversión



Tema 4: análisis comparativo del rendimiento

Tiempo de ejecución del Programa

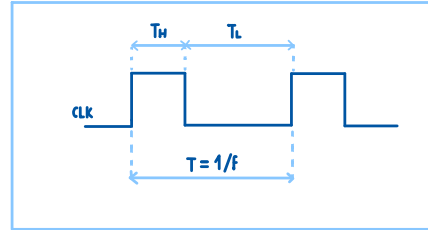
$$T_{\text{Ejec}} = N_i \cdot \text{CPI} \cdot T_{\text{Reloj}} = \frac{N_i \cdot \text{CPI}}{f_{\text{Reloj}}}$$

N_i = Número de instrucciones del programa

CPI = Ciclos por instrucción

f_{Reloj} = frecuencia del reloj

T_{Reloj} = Tiempo del reloj



MIPS (Millones de instrucciones por segundo)

$$\text{MIPS} = \frac{N_i}{T_{\text{Ejecución}} \cdot 10^6} = \frac{f_{\text{Reloj}}}{\text{CPI} \cdot 10^6}$$

MFLOPS (Millones de operaciones en coma flotante)

$$\text{MFLOPS} = \frac{\text{Operaciones de coma flotante realizadas}}{T_{\text{Ejecución}} \cdot 10^6}$$

Cada operación se multiplica por un peso proporcional a su complejidad:

→ ADD, SUB, COMPARE, MULT → 1 operación

→ DIVIDE, SQRT → 4 operaciones

→ EXP, SIN, ATAN → 8 operaciones

Índice spec

speedups

Es la media geométrica de las ganancias en velocidad con respecto a una máquina de referencia.

$$I_{\text{SPEC}} = \sqrt[n]{\frac{t_1^{\text{REF}}}{t_1} \cdot \frac{t_2^{\text{REF}}}{t_2} \cdot \dots \cdot \frac{t_n^{\text{REF}}}{t_n}} \quad n = 10$$

- SPEC (Máquina 1) > SPEC (Máquina 2)
 Media geométrica (Máquina 1) < Media geométrica (Máquina 2)

- Modo Base: mismas opciones de compilación y lenguaje
- Modo Peak: buscar opciones de compilación óptimas para cada programa

Media aritmética

Dado un conjunto un de n medidas, $t_1, \dots, t_n$, definimos su media aritmética:

$$\bar{t} = \frac{1}{n} \sum_{k=1}^n t_k$$

Media aritmética ponderada

Se puede asociar a cada medida t_k un peso w_k , obteniéndose la media aritmética ponderada:

$$\bar{t}_k = \sum_{k=1}^n w_k \cdot t_k \quad \text{con} \quad \sum_{k=1}^n w_k = 1$$

Si t_k es el tiempo de ejecución del programa de benchmark K -ésimo en la máquina a testar, w_k puede escogerse de forma inversamente proporcional a dicho tiempo de ejecución

$$w_k \equiv \frac{C}{t_{k, \text{REF}}}$$

$$C = \frac{1}{\sum_{k=1}^n \frac{1}{t_{k, \text{REF}}}}$$

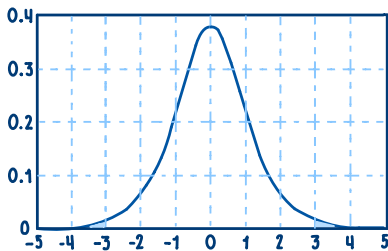
Media geométrica

Dado un conjunto un de n medidas, $S_1, \dots, S_n$, definimos su media geométrica:

$$\bar{S}_g = \sqrt[n]{\prod_{k=1}^n S_k} = \left(\prod_{k=1}^n S_k \right)^{1/n}$$

Nivel o grado de significatividad (α)

Distribución t de Student con 5 grados de libertad



$$P\text{-valor} = P(|t| \geq |t_{\text{exp}}|) \text{ en } T_{n-1} = 2 \cdot P(t \leq -|t_{\text{exp}}|) \text{ en } T_{n-1}$$

La probabilidad de obtener un valor de $|t|$ igual o superior a 2,99 de una distribución de T de Student con 5 grados de libertad es de 0,03 ($P\text{-valor} = 0,03$)

Para saber si es mucho o poco definiremos un umbral: nivel o grado de significatividad α

Si $P\text{-valor} < \alpha$ diremos que para un nivel de significatividad α o para un NIVEL DE CONFIANZA $(1-\alpha) \cdot 100$ las máquinas tienen rendimientos diferentes.
normalmente 95%

Intervalos de confianza para T_{EXP}

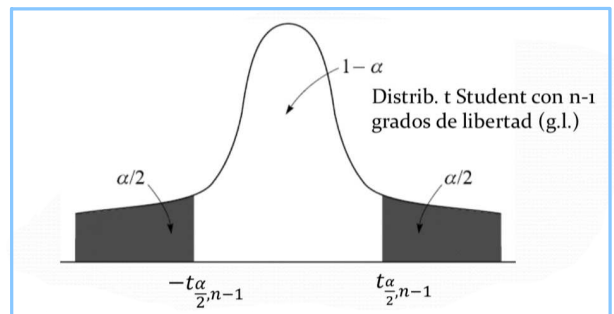
Para un nivel de significatividad α (normalmente 0,05 = 5%), buscamos el valor $t_{\frac{\alpha}{2}, n-1}$ que cumpla $\text{Prob}(|t| > t_{\frac{\alpha}{2}, n-1}) = \alpha$, o equivalentemente:

$$\text{Prob}\left(-t_{\frac{\alpha}{2}, n-1} \leq t \leq t_{\frac{\alpha}{2}, n-1}\right) = 1-\alpha$$

Para un nivel de confianza $1-\alpha$, para aceptar H_0 el valor de T_{exp} debe situarse en el intervalo:

$$\left[-t_{\frac{\alpha}{2}, n-1}, t_{\frac{\alpha}{2}, n-1}\right]$$

INTERVALO DE CONFIANZA



H_0 : suponemos mismo rendimiento

Intervalos de confianza para D_{REAL}

$$t_{\text{exp}} = \frac{\bar{d} - \bar{d}_{\text{REAL}}}{s/\sqrt{n}} \in \left[-t_{\frac{\alpha}{2}, n-1}, t_{\frac{\alpha}{2}, n-1}\right]$$



$$\bar{d}_{\text{REAL}} \in \left[\bar{d} - \frac{s}{\sqrt{n}} \cdot t_{\frac{\alpha}{2}, n-1}, \bar{d} + \frac{s}{\sqrt{n}} \cdot t_{\frac{\alpha}{2}, n-1}\right]$$

Análisis de la varianza de un factor (ANOVA)

$$\text{Modelo: } y_{ij} = m_{\text{global}} + \varepsilon_j + R_{ij}$$

$y_{ij} \longrightarrow$ Observaciones (tiempos de ejecución obtenidos en cada prueba)

$m_{\text{global}} \longrightarrow$ media global de todas las observaciones

$$m_{\text{global}} = \frac{1}{n_{\text{rep}} \cdot n_{\text{niv}}} \sum_{i=1}^{n_{\text{rep}}} \sum_{j=1}^{n_{\text{niv}}} y_{ij}$$

- 1) varianza constante
- 2) distribución normal



$R_{ij} \longrightarrow$ perturbaciones o error experimental (ruido)

$\varepsilon_j \longrightarrow$ efecto debido al nivel j -ésimo

$$\varepsilon_j = \frac{1}{n_{\text{rep}}} \sum_{i=1}^{n_{\text{rep}}} y_{ij} - m_{\text{global}}$$

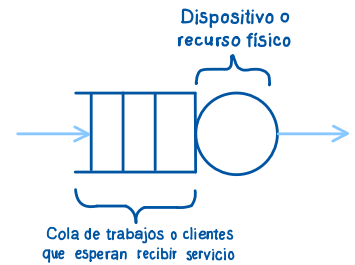
Tema 5: optimización del rendimiento de un servidor mediante análisis operacional

$$R = W + S$$

W = tiempo de espera en cola (waiting time)

S = tiempo de servicio (service time)

R = tiempo de respuesta (response time)



Cuando hay estaciones con infinitos servidores el tiempo de espera desaparece, por lo que $R = S$

Red cerrada de tipo batch

$$N_T = N_0$$

Red cerrada de tipo interactivo

$$N_T = N_0 + N_z$$

N_0 : número de trabajos en el servidor

N_z : número de trabajos en reflexión

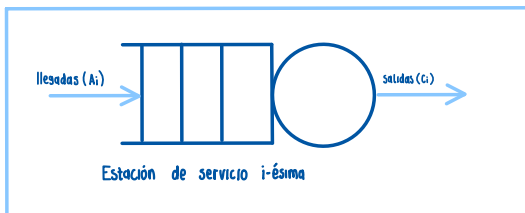
N_T : Red

T → tiempo de monitorización

C_i → salidas

A_i → llegadas o arrivals

B_i → tiempo ocupado o busy time



λ_i : tasa media de llegada (trabajos/segundo)

X_i : productividad media (trabajos/segundo)

U_i : utilización media (sin unidades)

S_i : tiempo medio de servicio (segundos [/trabajo])

W_i : tiempo medio de espera en cola (segundos [/trabajo])

R_i : tiempo medio de respuesta (segundos [/trabajo])

Tasa de llegada

$$\lambda_i = \frac{A_i}{T}$$

Productividad

$$X_i = \frac{C_i}{T}$$

Utilización

$$U_i = \frac{B_i}{T}$$

Tiempo de servicio

$$S_i = \frac{B_i}{C_i}$$

Tiempo de respuesta

$$R_i = W_i + S_i$$

N_i : número medio de trabajos en la estación de servicio

Q_i : número medio de trabajos en la cola de espera

$$U_i = N_i - Q_i$$

→

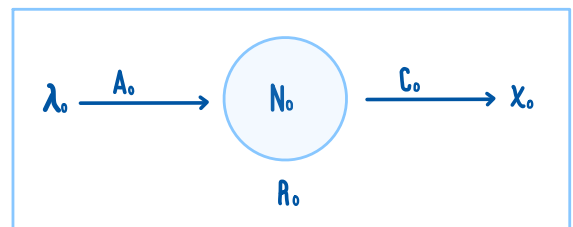
$$N_i = Q_i + U_i$$

V_i : razón media de visita

D_i : demanda media de servicio

$$V_i = \frac{C_i}{C_0}$$

$$D_i = \frac{B_i}{B_0} = V_i \cdot S_i$$



Hipótesis del equilibrio de flujo

$$C_o \approx A_o \quad \Leftrightarrow \quad X_o \approx \lambda_o \quad \longrightarrow \quad C_i \approx A_i, X_i \approx \lambda_i, \forall i = 1..k$$

Ley de Little

$$N_o = \lambda_o \cdot R_o = X_o \cdot R_o \quad \longrightarrow \quad \text{Solo funciona cuando el servidor está en equilibrio de flujo}$$

$$N_i = \lambda_i \cdot R_i = X_i \cdot R_i \quad \longrightarrow \quad Q_i = \lambda_i \cdot W_i = X_i \cdot W_i$$

Ley de la utilización

$$V_i = X_i \cdot S_i = \lambda_o \cdot S_i \quad \longrightarrow \quad \text{Relaciona la productividad media con el tiempo de servicio medio}$$

$$S_i = \frac{B_i}{C_i} = \frac{B_i/T}{C_i/T} = \frac{U_i}{X_i} \quad \longrightarrow \quad \text{Demostración}$$

Ley del Flujo Forzado y relación utilización - demanda

$$\text{Ley del Flujo forzado} \quad \longrightarrow \quad X_i = X_o \cdot V_i = \lambda_o \cdot V_i \quad \longrightarrow \quad V_i = \frac{C_i}{C_o} = \frac{C_i/T}{C_o/T} = \frac{X_i}{X_o} \quad \longrightarrow \quad \text{Demostración}$$

$$\text{Relación Utilización - Demanda de servicio} \quad \longrightarrow \quad U_i = X_o \cdot D_i = \lambda_o \cdot D_i \quad \longrightarrow \quad D_i = \frac{B_i}{C_o} = \frac{B_i/T}{C_o/T} = \frac{U_i}{X_o} \quad \longrightarrow \quad \text{Demostración}$$

Ley general del tiempo de respuesta

$$R_o = V_1 \cdot R_1 + V_2 \cdot R_2 + \dots + V_k \cdot R_k = \sum_{i=1}^k V_i \cdot R_i$$

Ley del tiempo de respuesta iterativo

$$N_t = N_z + N_o = X_o \cdot Z + X_o \cdot R_o = X_o \cdot (Z + R_o) \quad \longrightarrow \quad R_o = \frac{N_r}{X_o} - z$$

Identificación del cuello de botella

$$U_i = X_o \cdot D_i = \lambda_o \cdot D_i$$

Denotaremos por "b" (bottleneck) al índice del dispositivo cuello de botella. Su demanda de servicio y su utilización vendrán dadas por :

$$D_b = \max_{i=1..k} \{D_i\} = V_b \cdot S_b$$

$$U_b = \max_{i=1..k} \{U_i\} = X_o \cdot D_b$$

Saturación del servidor

En saturación, el cuello de botella está al máximo de su productividad :

$$1 = U_b = X_o \cdot S_b$$

$$\longrightarrow X_o = 1/S_b$$

servidor equilibrado (balanced system)

$$U_i \approx U_j \quad \forall i,j = 1..k$$

$$D_i \approx D_j \quad \forall i,j = 1..k$$

Límites optimistas en redes abiertas

$$R_o \longrightarrow R_o^{\min} = \sum_{i=1}^k V_i \cdot S_i = \sum_{i=1}^k D_i \equiv D$$

Valor optimista del tiempo medio de respuesta

$$\text{Si } U_b = 1 \Rightarrow X_o^{\max} = \frac{1}{D_b}$$

Valor máximo de la productividad

Límites optimistas en redes cerradas

Para valores de carga altos

Ley de Little a la red completa ($N_r = N_o + N_z$)

$$N_r = X_o \cdot (R_o + Z)$$

$$R_o = \frac{N_r}{X_o} - Z$$

Valor optimista de la productividad

$$\text{Si } U_b \longrightarrow 1 \Rightarrow X_o \longrightarrow X_o^{\max} = \frac{1}{D_b}$$

Valor optimista del tiempo de respuesta

$$R_o \longrightarrow \left(\frac{N_r}{X_o^{\max}} \right) - Z = D_b \cdot N_r - Z$$

Para valores de carga bajos

Ley de Little a la red completa ($N_r = N_o + N_z$)

$$N_r = X_o \cdot (R_o + Z)$$

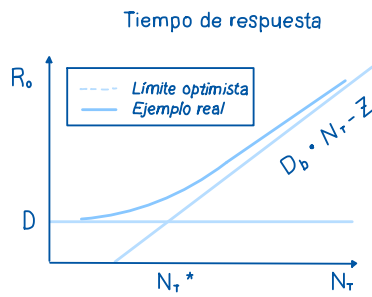
$$X_o = \frac{N_r}{R_o + Z}$$

Valor optimista de la productividad

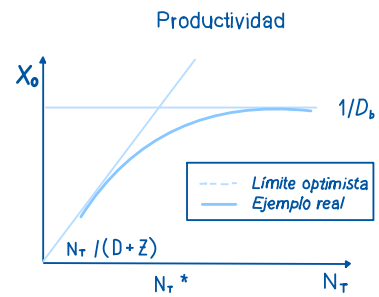
$$X_o \longrightarrow \frac{N_r}{R_o^{\min} + Z} = \frac{N_r}{D + Z}$$

Valor optimista del tiempo de respuesta

$$R_o \longrightarrow R_o^{\min} = \sum_{i=1}^k V_i \cdot S_i = \sum_{i=1}^k D_i \equiv D$$



$$R_o \geq \max \{ D, D_b \cdot N_r - Z \}$$



$$X_o \leq \min \left\{ \frac{N_r}{D + Z}, \frac{1}{D_b} \right\}$$

Punto teórico de saturación (knee-point)

$$D = D_b \cdot N_r^* - Z \Rightarrow N_r^* = \frac{D+Z}{D_b}$$

No es una
Ley Operacional

Hipótesis de la independencia en la llegada de trabajos en red abierta

El momento en el que llega un trabajo es independiente de cuando llegó el trabajo anterior

$$R_i = X_i \cdot R_i \cdot S_i + S_i \Rightarrow R_i = \frac{S_i}{1 - U_i} = \frac{S_i}{1 - X_o \cdot D_i} = \frac{S_i}{1 - \lambda_o \cdot V_i \cdot S_i}$$